

PROGRAMMENTWURF · PROTOKOLL

LeagueMaster

Eine Anwendung zur Verwaltung von Liga- und Turnierspielen

AUTOREN

Leon Rother · Moritz Mürle

MATRIKELNUMMER

[MNR]

ABGABEDATUM

03.05.2026

Inhaltsverzeichnis

01 **Einführung**
Übersicht · Start · Technik

02 **Softwarearchitektur**
Clean Architecture · Domain · Dependency Rule

03 **SOLID**
SRP · OCP · DIP

04 **Weitere Prinzipien**
GRASP · DRY

05 **Unit Tests**
ATRIP · Fakes & Mocks

06 **Domain Driven Design**
Ubiquitous Language · Aggregates

07 **Refactoring**
Code Smells · Refactorings

08 **Entwurfsmuster**
Adapter · MVC

KAPITEL

01

Einführung

Übersicht über die Applikation · Starten · Technischer Überblick

Übersicht über die Applikation

01

Was macht LeagueMaster?

Eine kommandozeilenbasierte (CLI) Turnierverwaltungsanwendung. Sie organisiert sportliche Wettbewerbe in zwei Formaten: als klassische Liga (Jeder-gegen-Jeden, Spieltage, Tabelle) und als Knockout-Turnier (KO-System mit Bracket bis zum Finale).

02

Welches Problem löst sie?

Turnierorganisation umfasst viele wiederkehrende, fehleranfällige Schritte: Teams erfassen, Paarungen generieren, Ergebnisse eintragen, Tabellen aktuell halten. LeagueMaster automatisiert Spielplanerstellung (Round-Robin, Bracket), erzwingt Formatregeln und zeigt Tabellen/Brackets nach jedem Ergebnis an.

03

Wie funktioniert sie?

Nach dem Start begrüßt die Anwendung den Nutzer mit einer Modusauswahl. Es folgt ein geführter Workflow per Texteingabe, vom Anlegen über Teams hinzufügen bis zum Eintragen von Ergebnissen. Nach jedem Befehl gibt es direktes Feedback und der nächste Schritt wird angezeigt.

Starten der Applikation

Zwei Wege zum Start: per Terminal (empfohlen) oder per Doppelklick auf dem Desktop.

Einmalige Vorbereitung

```
chmod +x linux/run-leaguemaster.sh  
chmod +x linux/LeagueMaster.desktop
```

Starten im Terminal

```
./linux/run-leaguemaster.sh
```

Starten per Doppelklick

1. LeagueMaster.desktop doppelklicken
2. Bei Fedora: "Datei als Anwendung starten" erlauben

Technischer Überblick

Verwendete Technologien

Voraussetzung	Version	Zweck
Java (OpenJDK)	21 (LTS)	Laufzeitumgebung
Apache Maven	3.x	Build-Werkzeug zum Kompilieren und Packen

Java 21 (LTS)

Laufzeitumgebung

Apache Maven 3

Build-Werkzeug zum Kompilieren und Packen

JUnit 5 (Jupiter 5.10.2)

Test-Framework

KAPITEL

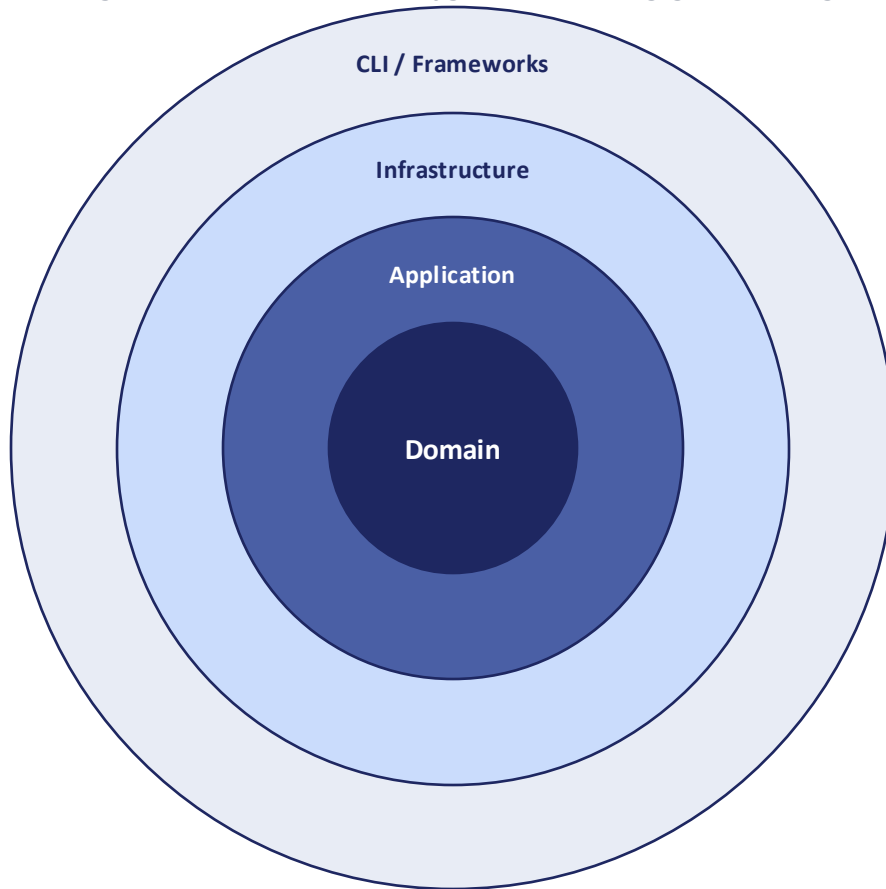
02

Softwarearchitektur

Clean Architecture · Domain Code · Dependency Rule

Clean Architecture

Die Anwendung ist in vier Schichten aufgeteilt. Abhängigkeiten zeigen ausschließlich nach innen.



1 · Domain

League, Team, Match, Score, LeagueRepository (Interface)

2 · Application

CreateLeagueService, AddTeamService, RecordMatchResultService, DTOs

3 · Infrastructure

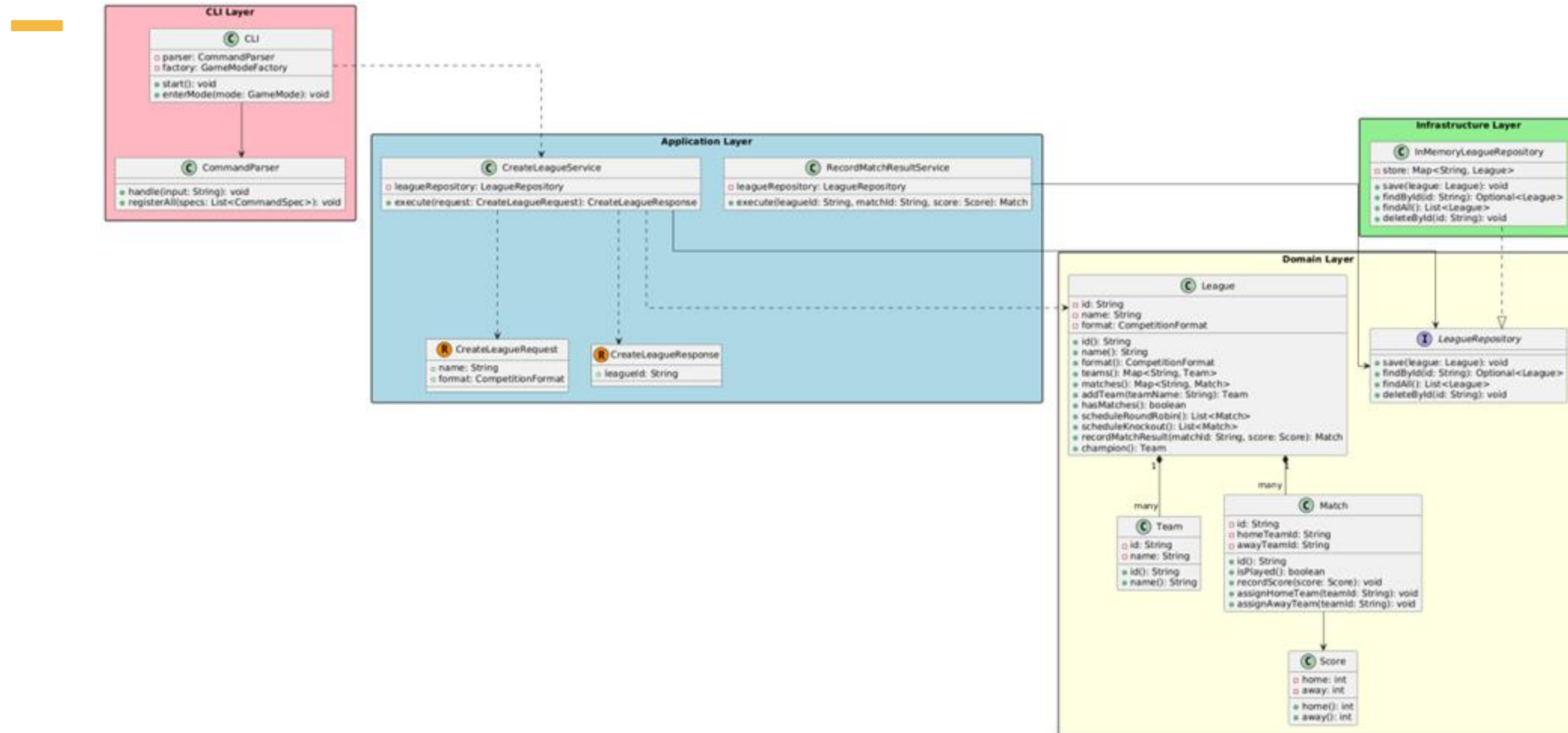
InMemoryLeagueRepository, Logger

4 · CLI / Presentation

CLI, CommandParser, alle Commands, GameModes, Renderer

→ Abhängigkeiten zeigen nach innen

UML – Klassendiagramm der Architektur



Übersicht aller vier Schichten mit ihren Hauptklassen und Abhängigkeiten.

Domain Code



Definition

Domain Code bezeichnet den Teil der Anwendung, der ausschließlich fachliche Logik enthält — also ohne jede Abhängigkeit zu Frameworks, Datenbanken oder Benutzeroberflächen.

Beispielklasse: Match

Die Klasse Match aus dem Domain-Layer ist ein typisches Beispiel: Sie verwaltet ihre Teilnehmer (homeTeamId, awayTeamId), nimmt Ergebnisse entgegen und prüft fachliche Regeln (z. B. „kein doppeltes Eintragen eines Ergebnisses“, „Match muss vollständig besetzt sein“). Sie kennt keine Datenbank, kein Framework, keine UI — nur reine Domänenlogik.

Match.java — Beispiel für reinen Domain Code

Keine Import statements.

```
package de.leaguemaster.domain.model;

public class Match {
    private final String id;
    private String homeTeamId;
    private String awayTeamId;
    private Score score;

    public Match(String id, String homeTeamId, String awayTeamId) {
        this.id = id;
        this.homeTeamId = homeTeamId;
        this.awayTeamId = awayTeamId;
    }
}
```

Konforme Klasse: CreateLeagueService

DEPENDENCY RULE: ERFÜLLT

CreateLeagueService

Application-Layer

Hängt ab von (CreateLeagueService → ...)

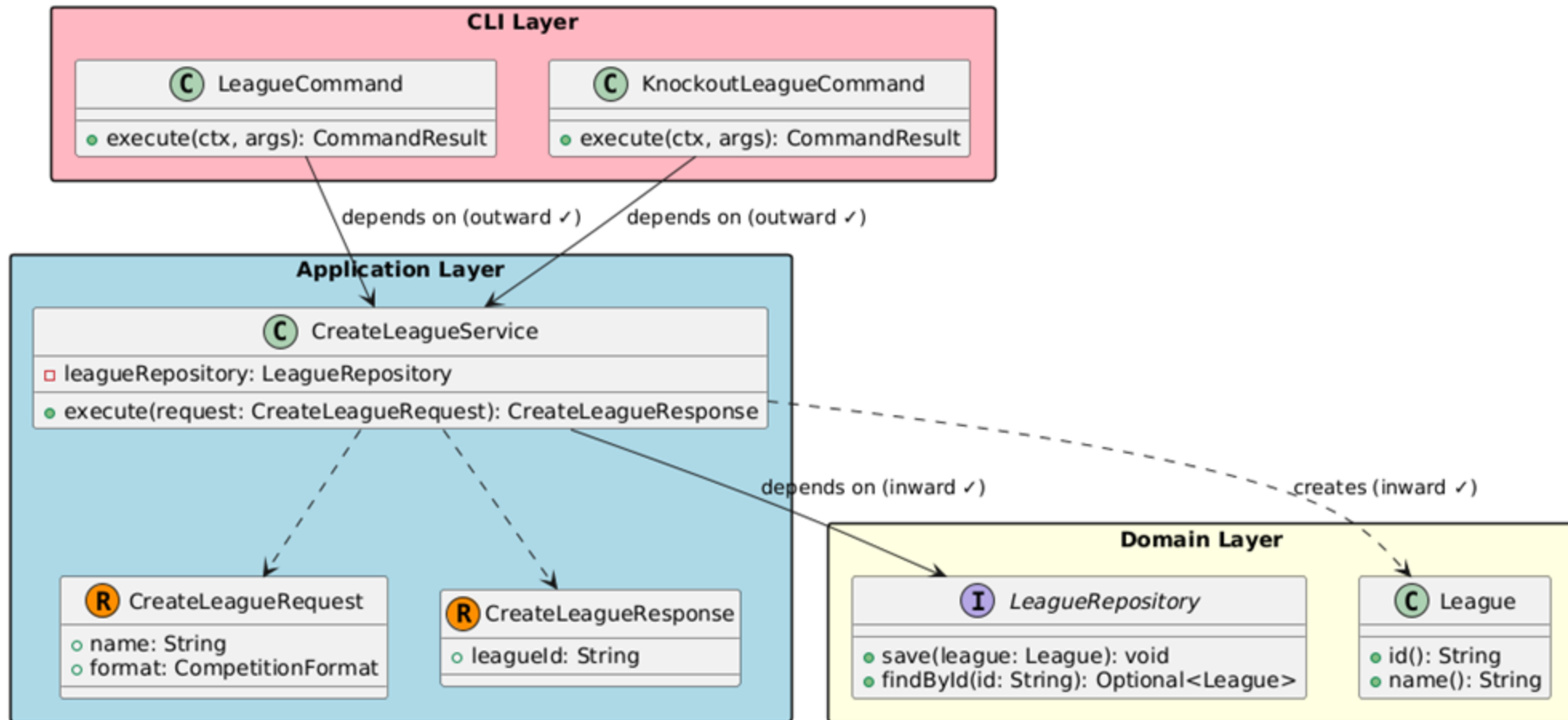
- LeagueRepository — Interface aus Domain (nie konkrete Implementierung)
- League — Domain-Entität
- CreateLeagueRequest, CreateLeagueResponse — DTOs der gleichen Schicht

Wird genutzt von (... → CreateLeagueService)

- LeagueCommand (CLI-Layer)
- KnockoutLeagueCommand (CLI-Layer)
- *Beide nutzen den Service als Application-Use-Case.*

✓ Alle Abhängigkeiten zeigen nach innen: Application → Domain. CLI → Application. Regel erfüllt.

UML: CreateLeagueService — Abhängigkeiten zeigen nach innen



CLI → Application → Domain — die korrekte Abhängigkeitsrichtung.

Verletzende Klasse: GameModeSelectionService

DEPENDENCY RULE: VERLETZT

GameModeSelectionService

Application-Layer (*de.leaguemaster.application.usecase*)

Hängt ab von ⚠ Verstoß

Application → CLI (äußere Schicht!)

- GameModeFactory (CLI)
- GameModeType (CLI)
- SelectModeCommand (CLI)
- CommandSpec (CLI)

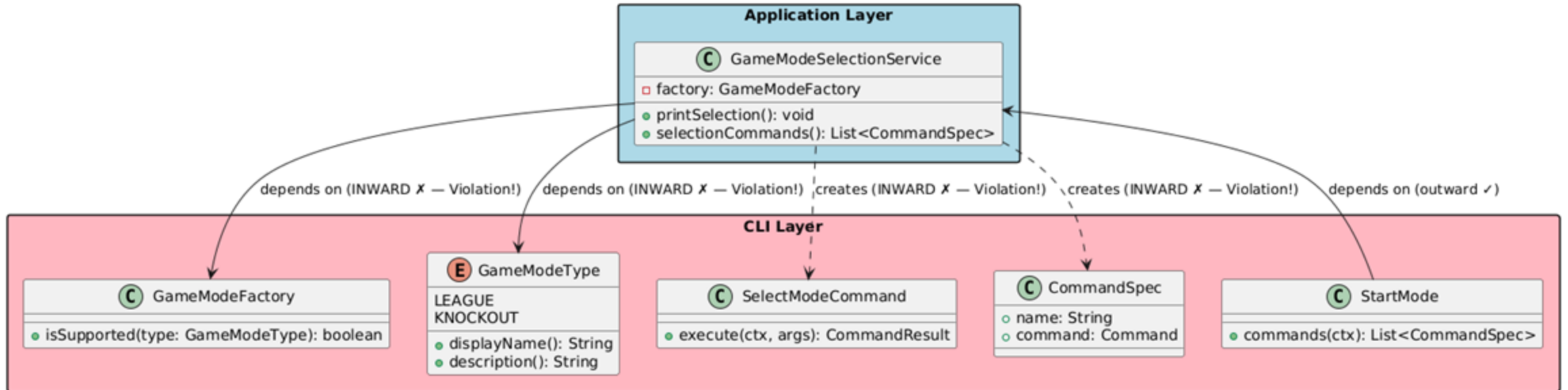
Wird genutzt von

CLI → Application (korrekte Richtung)

- StartMode (CLI-Layer) ruft selectionCommands() auf.
- *Diese Richtung ist regelkonform — das Problem liegt in der entgegengesetzten Richtung des Service selbst.*

Fazit: Der Service zeigt nach außen in die CLI-Schicht. Das verletzt die Dependency Rule.

UML: GameModeSelectionService — Abhängigkeiten zeigen nach außen



Vier Pfeile aus Application → CLI markieren die Verletzungen der Dependency Rule.

KAPITEL

03

SOLID

Single Responsibility · Open/Closed · Dependency Inversion

Positives Beispiel: RecordMatchResultService

SRP: ERFÜLLT

Single Responsibility Principle

Eine Klasse soll genau einen Grund zur Änderung haben.

Aufgabe der Klasse

RecordMatchResultService hat genau eine Verantwortlichkeit: ein Spielergebnis in einer bestehenden Liga eintragen und persistieren. Die Klasse lädt die Liga, delegiert die fachliche Validierung an das Domain-Objekt League und speichert den neuen Zustand.

Es gibt genau einen Grund, diese Klasse zu ändern: wenn sich die Logik des Eintragens eines Spielergebnisses ändert.

RecordMatchResultService — Code

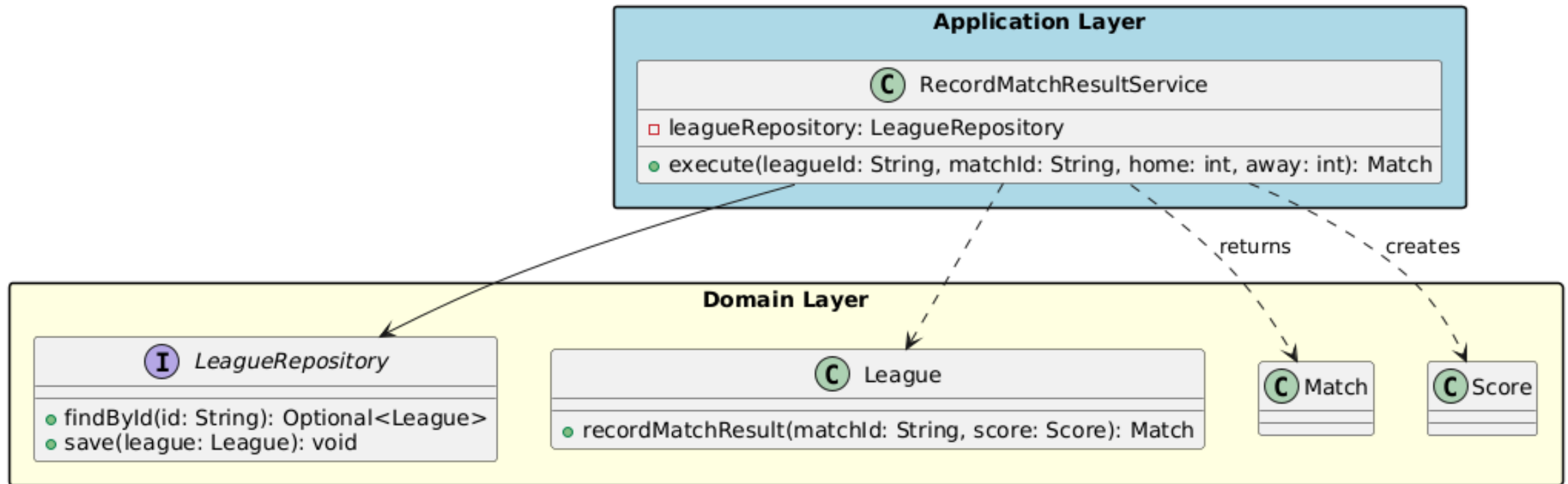
```
package de.leaguemaster.application.usecase;

public class RecordMatchResultService {
    private final LeagueRepository leagueRepository;

    public RecordMatchResultService(LeagueRepository leagueRepository) {
        this.leagueRepository = leagueRepository;
    }

    public Match execute(String leagueId, String matchId, int home, int away) {
        Optional<League> leagueOpt = leagueRepository.findById(leagueId);
        if (leagueOpt.isEmpty()) {
            throw new IllegalArgumentException("Liga nicht gefunden.");
        }
        League league = leagueOpt.get();
        Match match = league.recordMatchResult(matchId, new Score(home, away));
        leagueRepository.save(league);
        return match;
    }
}
```

RecordMatchResultService — UML



Genau eine Verantwortung: Liga laden, Ergebnis eintragen, speichern.

Negatives Beispiel: League

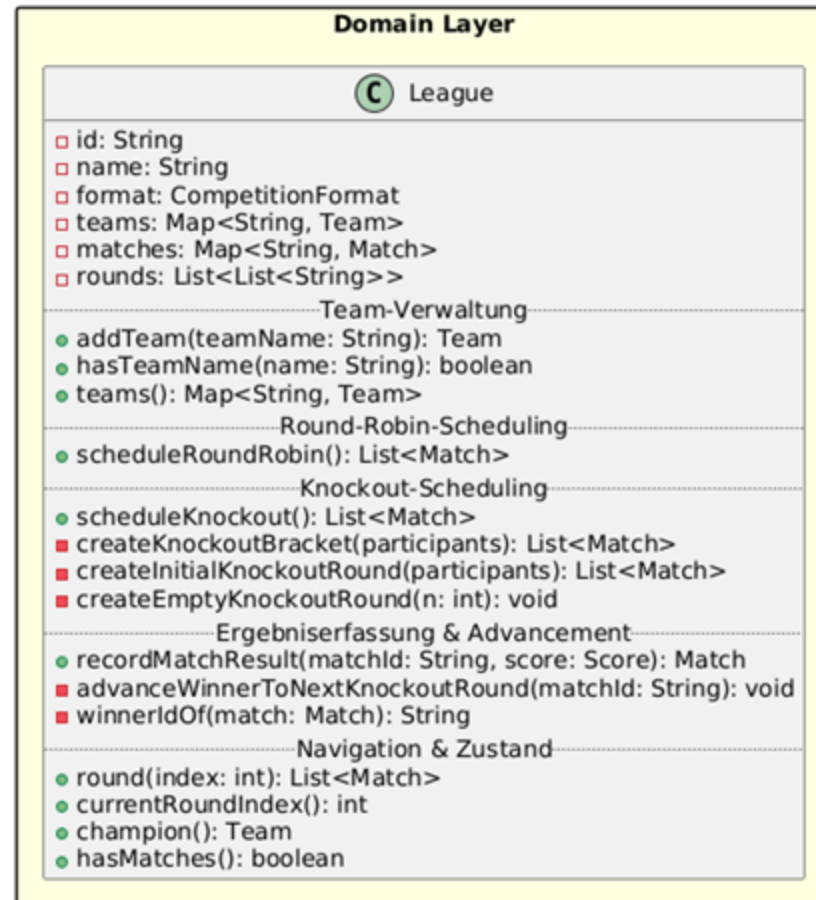
SRP: VERLETZT

League ist das Aggregate Root des Domain-Layers, übernimmt aber fünf Verantwortlichkeiten in einer Klasse:

1. Team-Verwaltung · `addTeam()`, `hasTeamName()`, `teams()`
2. Round-Robin-Spielplan · `scheduleRoundRobin()` (inkl. Rotationsalgorithmus)
3. Knockout-Bracket-Erstellung · `scheduleKnockout()`, `createKnockoutBracket()`, ...
4. Ergebniserfassung & Bracket-Advancement · `recordMatchResult()`, `advanceWinnerToNextKnockoutRound()`, `winnerIdOf()`
5. Zustandsabfragen & Navigation · `round()`, `currentRoundIndex()`, `totalRounds()`, `hasMatches()`, `champion()`

Jede dieser fünf Verantwortlichkeiten könnte sich aus eigenen Gründen ändern — ein klarer SRP-Verstoß.

League — fünf Verantwortlichkeiten in einer Klasse



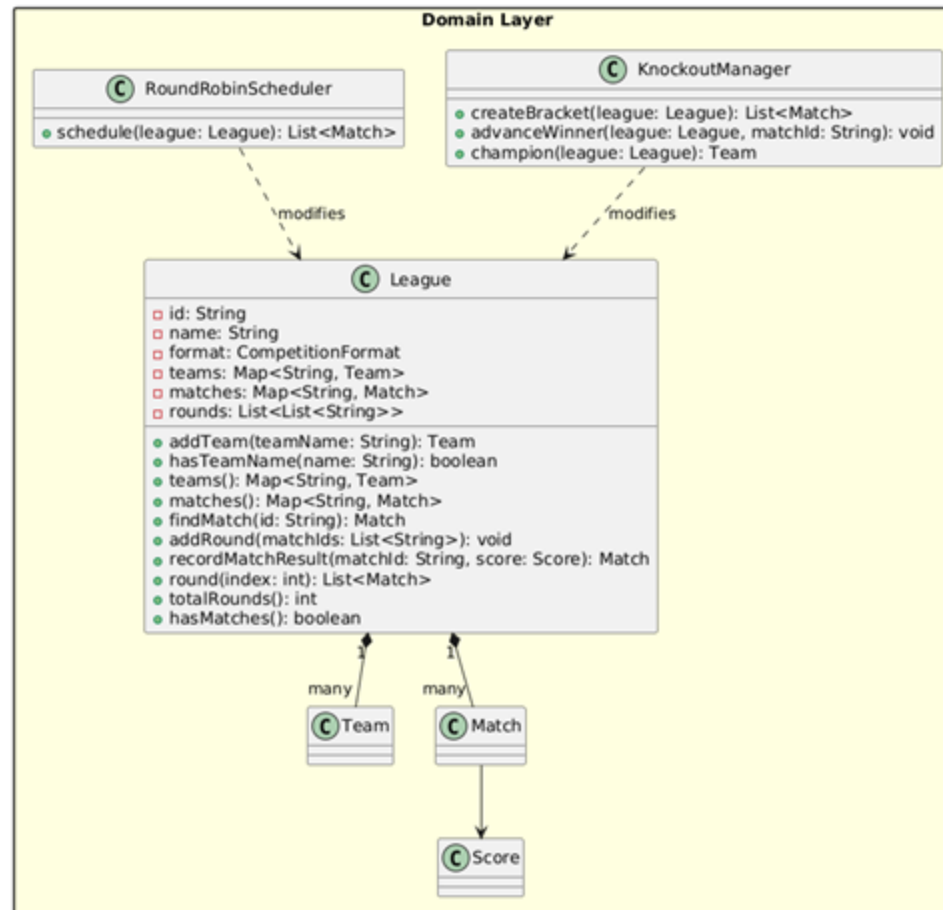
Die Methoden sind nach Verantwortlichkeit gruppiert. Jede Gruppe ist ein eigener Grund zur Änderung.

Verantwortlichkeitstabelle

#	Verantwortlichkeit	Betroffene Methoden
1	Team-Verwaltung	<code>addTeam()</code> , <code>hasTeamName()</code> , <code>teams()</code>
2	Round-Robin-Spielplan	<code>scheduleRoundRobin()</code> (inkl. Rotationsalgorithmus)
3	Knockout-Bracket-Erstellung	<code>scheduleKnockout()</code> , <code>createKnockoutBracket()</code> , <code>createInitialKnockoutRound()</code> , <code>createEmptyKnockoutRound()</code>
4	Ergebniserfassung & Bracket-Advancement	<code>recordMatchResult()</code> , <code>advanceWinnerToNextKnockoutRound()</code> , <code>winnerIdOf()</code>
5	Zustandsabfragen & Navigation	<code>round()</code> , <code>currentRoundIndex()</code> , <code>totalRounds()</code> , <code>hasMatches()</code> , <code>champion()</code>

Auflistung der fünf Verantwortlichkeiten und ihrer betroffenen Methoden.

Lösung: Verantwortlichkeiten extrahieren



RoundRobinScheduler und KnockoutManager nehmen League jeweils eine Verantwortung ab.

Positives Beispiel: CLI

OCP: ERFÜLLT

Open/Closed Principle

Eine Klasse soll offen für Erweiterung, aber geschlossen für Modifikation sein. Neues Verhalten soll durch Hinzufügen von Code möglich sein, ohne bestehenden Code zu ändern.

Warum OCP erfüllt

Um einen neuen Spielmodus (z. B. SwissMode) hinzuzufügen, wird ausschließlich eine neue Klasse SwissMode implements GameMode erstellt. CLI, CommandParser und alle bestehenden Modes bleiben unverändert.

Das Problem, das OCP hier löst:

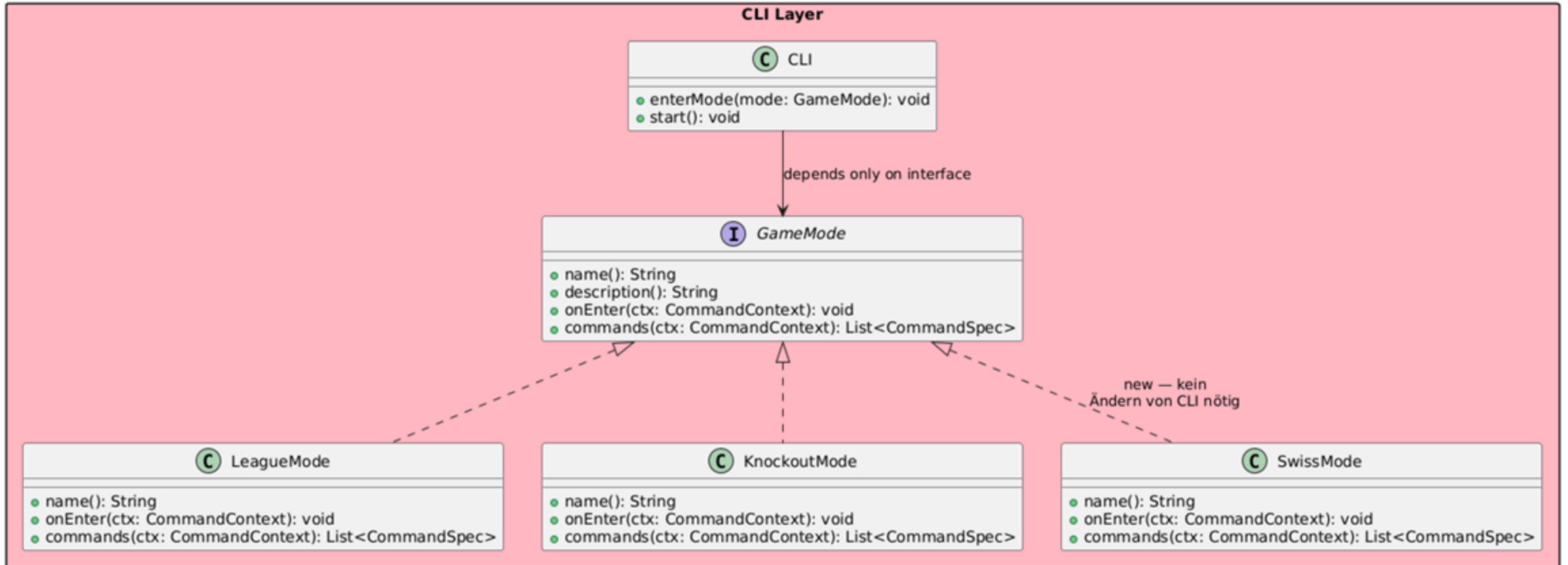
Ohne das Interface müsste CLI für jeden neuen Modus angepasst werden — bei wachsender Modusliste würde CLI immer komplexer und fehleranfälliger.

CLI.java — Code

```
public interface Command {  
  
    CommandResult execute(CommandContext ctx, CommandArgs args);  
  
}  
  
public interface GameMode {  
    List<CommandSpec> commands(CommandContext context);  
    // ...  
}  
public void enterMode(GameMode mode) {  
    parser.clearRegistry();  
    mode.onEnter(parser.context());  
    parser.registerAll(baseCommands());  
    parser.registerAll(mode.commands(parser.context())); // ← kennt keinen konkreten Modus  
}
```

CLI und CommandRegistry wissen nichts über konkrete Modi oder Befehle. Ein neuer Modus entsteht durch eine neue GameMode-Implementierung, ein neuer Befehl durch eine neue Command-Implementierung — ohne eine Zeile in CLI, CommandParser oder CommandRegistry zu ändern. Das ist OCP: offen für Erweiterung durch neue Klassen, geschlossen für Modifikation bestehender.

CLI hängt nur vom GameMode-Interface ab



Neue Modes erweitern, ohne dass CLI angefasst werden muss.

Negatives Beispiel: GameModeFactory

OCP: VERLETZT

Warum OCP verletzt

Jedes Hinzufügen eines neuen Spielmodus (z. B. SWISS) erfordert zwingend die Modifikation von GameModeFactory an zwei Stellen: dem switch-Block in create() und dem Vergleich in isSupported(). Die Klasse ist damit nicht geschlossen für Modifikation.

Lösungsweg

Statt des switch-Blocks wird ein Registrierungs-Mechanismus eingeführt. Ein neues Interface GameModeCreator kapselt die Erzeugungslogik pro Modus. GameModeFactory arbeitet nur noch gegen diese Abstraktion — sie muss nie wieder modifiziert werden.

GameModeFactory — Code mit switch-Block

```
public class GameModeFactory {
    // ... Services als Felder

    public boolean isSupported(GameModeType type) {

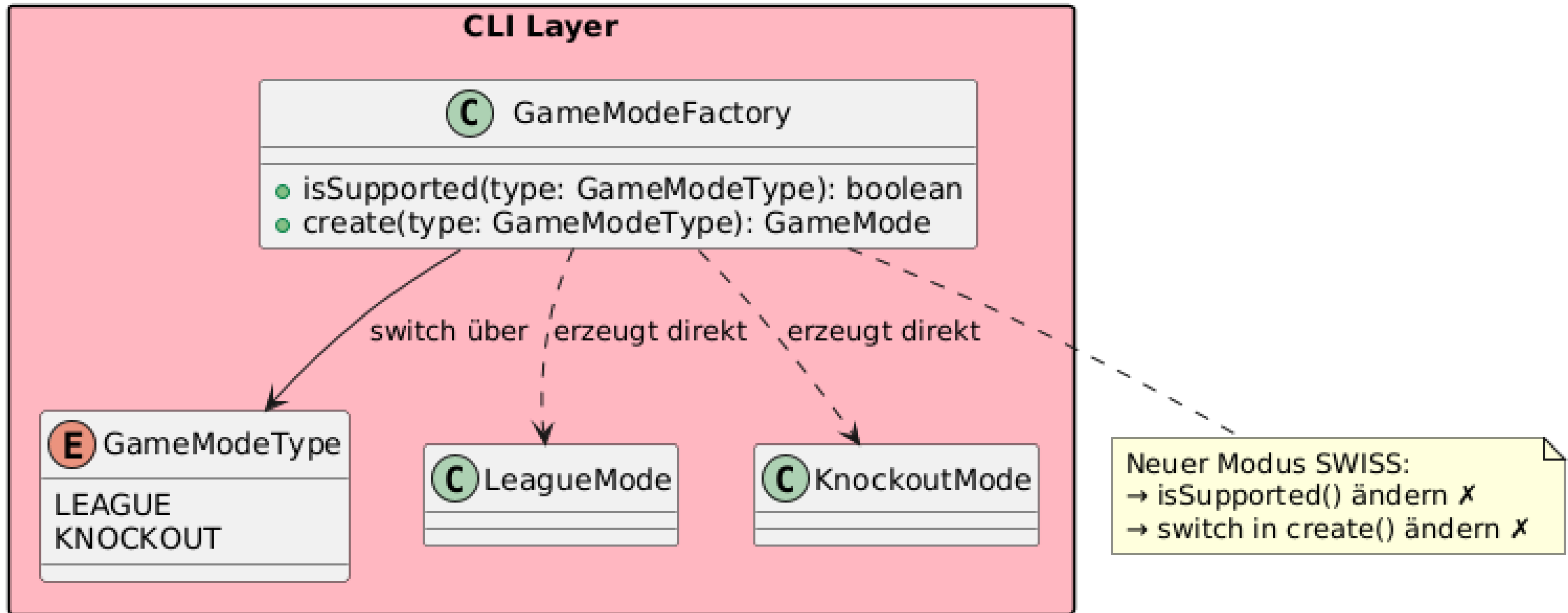
        // Ⓢ muss bei jedem neuen Modus erweitert werden

        return type == GameModeType.LEAGUE || type == GameModeType.KNOCKOUT;
    }

    public GameMode create(GameModeType type) {
        switch (type) {
            // Ⓢ muss bei jedem neuen Modus um einen case erweitert werden
            case LEAGUE: return new LeagueMode(...);
            case KNOCKOUT: return new KnockoutMode(...);
            default: throw new IllegalArgumentException("Unbekannter Modus: " + type);
        }
    }
}
```

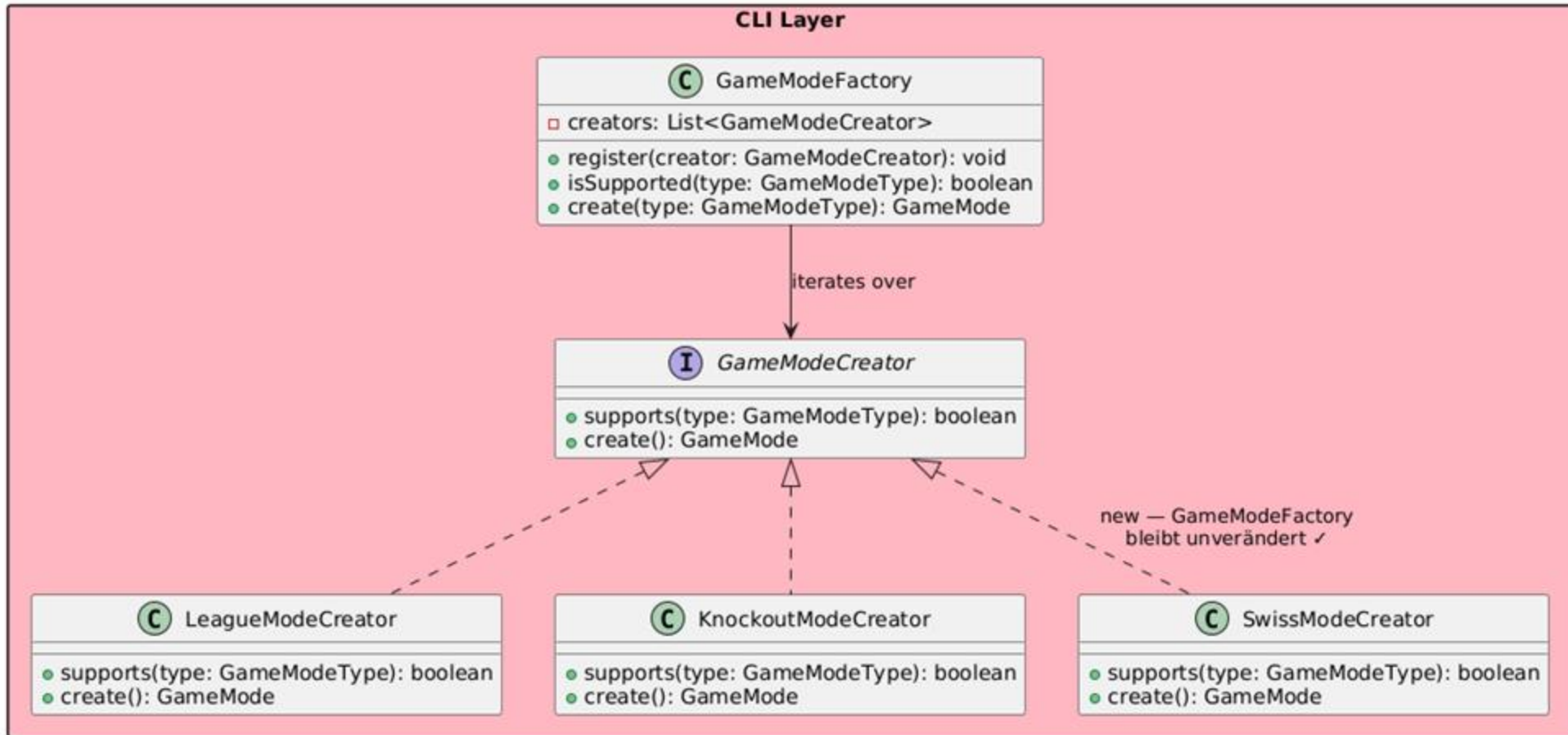
Der switch-Block in create() ist die Stelle, die bei jedem neuen Modus geändert werden müsste.

GameModeFactory — UML der Verletzung



Hinzufügen eines neuen Modus erfordert zwei Änderungen in GameModeFactory.

Lösung: Registry mit GameModeCreator



Neue Modes registrieren sich selbst — GameModeFactory bleibt unverändert.

Positives Beispiel: CreateLeagueService

DIP: ERFÜLLT

Dependency Inversion Principle

1. Module höherer Ebene sollen nicht von Modulen niedrigerer Ebene abhängen — beide sollen von Abstraktionen abhängen.
2. Abstraktionen sollen nicht von Details abhängen — Details sollen von Abstraktionen abhängen.

Begründung

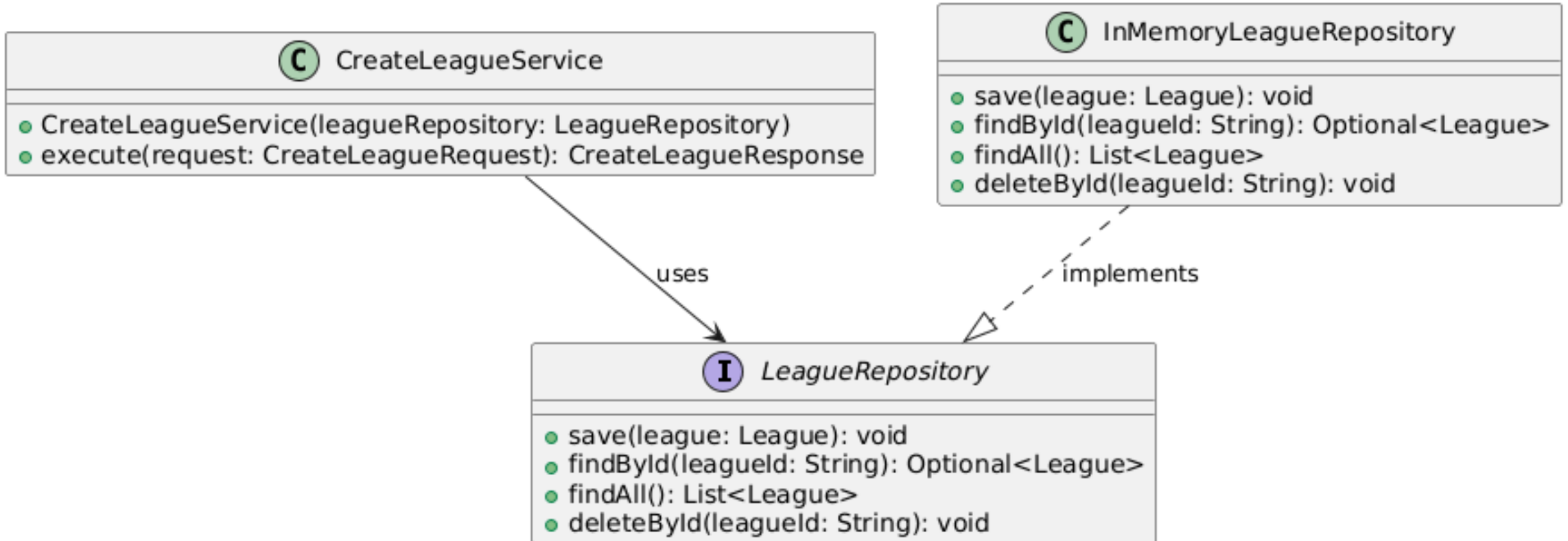
CreateLeagueService ist ein Modul der Anwendungsschicht (höhere Ebene). Anstatt direkt von der konkreten InMemoryLeagueRepository (niedrigere Ebene) abzuhängen, hängt es ausschließlich vom Interface LeagueRepository ab. Dieses Interface liegt im Domain-Layer. Die konkrete Klasse implementiert das Interface — beide Module hängen damit von der Abstraktion ab, nicht voneinander.

CreateLeagueService — Code

```
public class CreateLeagueService {  
    private final LeagueRepository leagueRepository; // Abhängigkeit auf Interface, NICHT auf  
    InMemoryLeagueRepository  
  
    public CreateLeagueService(LeagueRepository leagueRepository) {  
        this.leagueRepository = leagueRepository;  
    }  
  
    public CreateLeagueResponse execute(CreateLeagueRequest request) {  
        League league = new League(request.name(), format);  
        leagueRepository.save(league);  
        return new CreateLeagueResponse(league.id());  
    }  
}
```

Abhängigkeit auf Interface LeagueRepository, nicht auf InMemoryLeagueRepository.

CreateLeagueService — UML



Beide Module hängen von der Abstraktion ab, nicht voneinander.

Negatives Beispiel: MatchCommand

DIP: VERLETZT

Begründung

MatchCommand ist ein Modul der CLI-Schicht (höhere Ebene — Präsentation). Anstatt von einer Abstraktion abzuhängen, ruft es direkt statische Methoden der konkreten Klassen TableRenderer und WinnerRenderer auf. Diese sind konkrete Implementierungsdetails (niedrigere Ebene). Damit hängt das höhere Modul direkt vom niedrigeren ab — eine klare Verletzung von DIP.

Konsequenz

Soll das Darstellungsformat geändert werden (z. B. HTML statt Plaintext), muss MatchCommand selbst angepasst werden. Eine Austauschmöglichkeit ohne Code-Änderung am MatchCommand besteht nicht.

Lösung

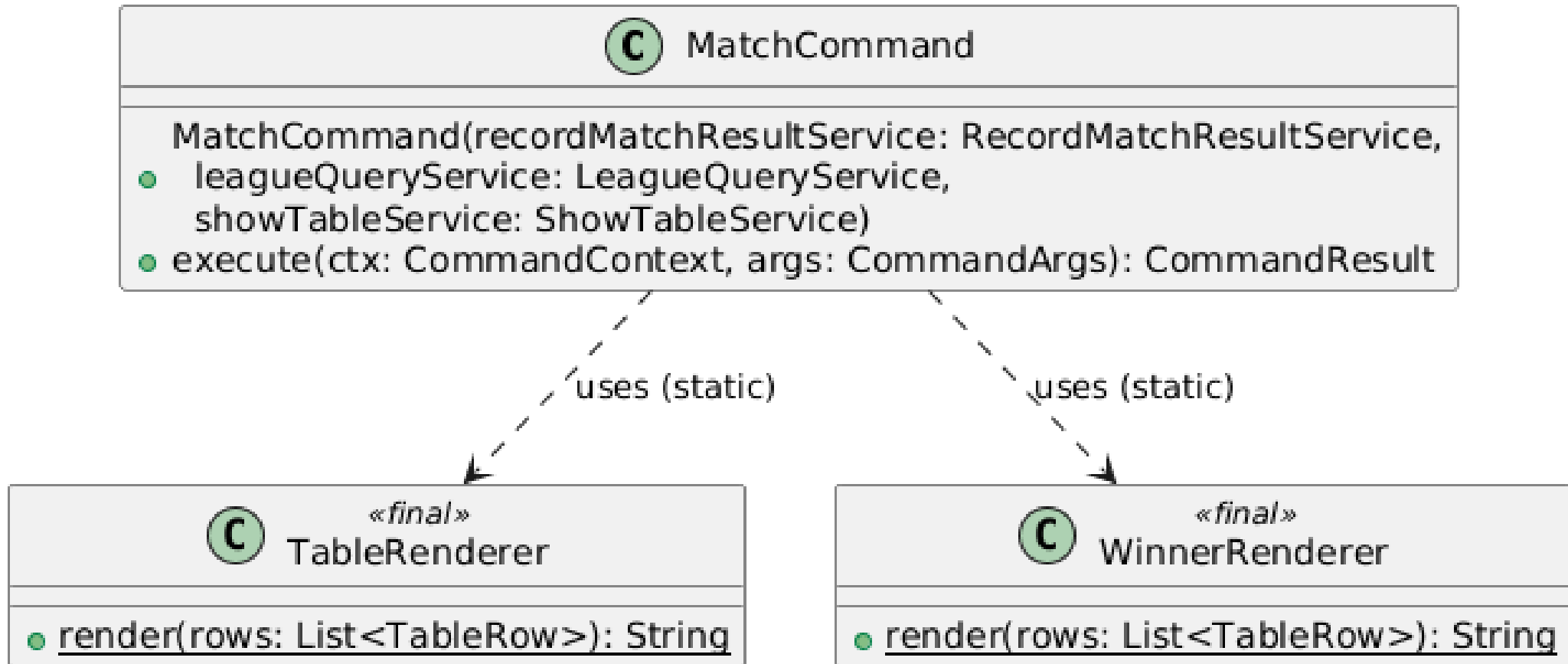
Es wird ein Interface MatchResultPresenter eingeführt. MatchCommand erhält dieses per Constructor Injection und kennt nur noch die Abstraktion. Die konkrete Klasse DefaultMatchResultPresenter implementiert das Interface und delegiert intern an TableRenderer und WinnerRenderer.

MatchCommand — Code mit direkten statischen Aufrufen

```
import de.leaguemaster.cli.output.TableRenderer; // direkte Abhängigkeit auf konkreten Typ
import de.leaguemaster.cli.output.WinnerRenderer; // direkte Abhängigkeit auf konkreten Typ

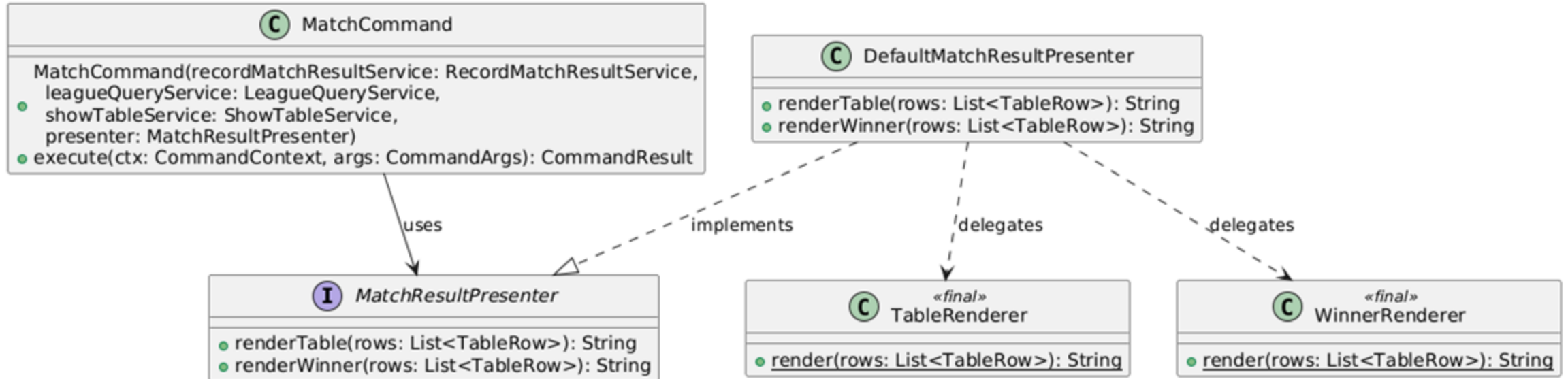
// ...
List<TableRow> rows = showTableService.execute(ctx.getCurrentLeagueId());
sb.append(TableRenderer.render(rows)); // statischer Aufruf auf konkrete Klasse
sb.append(WinnerRenderer.render(rows)); // statischer Aufruf auf konkrete Klasse
```

MatchCommand — UML der Verletzung



Statische Aufrufe auf konkrete Klassen statt Abhängigkeit auf eine Abstraktion.

Lösung: Interface MatchResultPresenter



Beide Module hängen jetzt von der Abstraktion ab, nicht voneinander.

KAPITEL

04

Weitere Prinzipien

GRASP · DRY

Geringe Kopplung — CommandArgs

Aufgabe der Klasse

CommandArgs parst eine rohe Nutzereingabe (String) in eine strukturierte Form: einen Befehlsnamen, Positionsargumente und benannte Optionen (--key value). Der interne Tokenizer unterstützt gequotete Werte (z. B. --name "FC Bayern").

Warum liegt geringe Kopplung vor?

Ausgehende Abhängigkeiten:

CommandArgs importiert ausschließlich `java.util.*` — keine einzige Domain-, Application- oder andere CLI-Klasse. Im UML-Diagramm zeigt kein Pfeil von CommandArgs auf eine Projektklasse.

Eingehende Abhängigkeiten:

CommandArgs wird breit genutzt: von CommandParser und allen 11 Command-Klassen. Die Klasse wird also breit genutzt, hängt aber selbst von nichts ab.

Konsequenz: Die Klasse wäre ohne eine einzige Änderung in einem völlig anderen Projekt einsetzbar, ein klares Zeichen maximal geringer Kopplung.

CommandArgs — Code

```
// CommandArgs.java – keine einzige Projekt-Klasse importiert
import java.util.*; // ausschließlich Java-Standardbibliothek

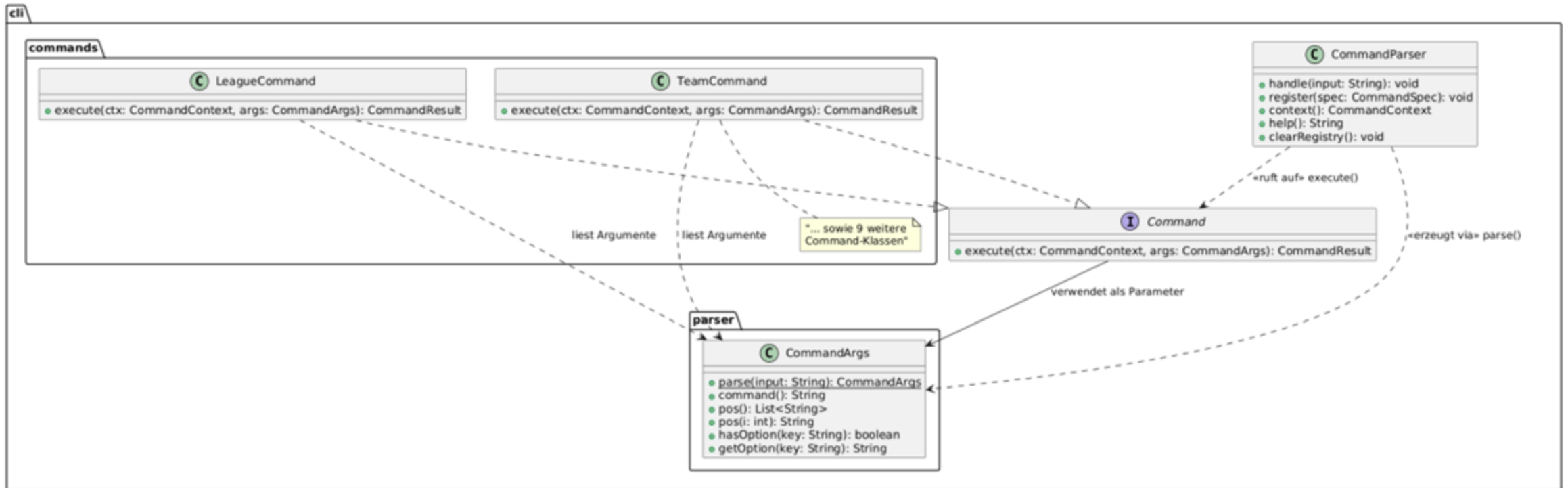
public class CommandArgs {
    private final String commandName;
    private final List<String> positionals = new ArrayList<>();
    private final Map<String, String> options = new HashMap<>();

    public static CommandArgs parse(String input) { ... }
    public String command() { return commandName; }
    public String pos(int i) { ... }
    public boolean hasOption(String key) { return options.containsKey(key); }
    public String getOption(String key) { return options.get(key); }
}

// CommandParser.java – erzeugt CommandArgs und reicht es weiter
CommandArgs args = CommandArgs.parse(input); // Zeile 57
registry.resolve(cmdName)
    .map(cmd -> cmd.execute(ctx, args)); // Zeile 61

// Beispiel-Nutzer: TeamCommand.java
public CommandResult execute(CommandContext ctx, CommandArgs args) {
    String action = args.pos(0); // Positionsargument lesen
    String name = args.getOption("name"); // benannte Option lesen
    ...
}
```

CommandArgs — UML



CommandArgs hängt von keiner Projektklasse ab; 11 Command-Klassen nutzen sie.

Polymorphismus — GameMode



GRASP Polymorphismus

Wenn sich Verhalten je nach Typ unterscheidet, soll diese Variation nicht durch bedingte Verzweigungen (if/switch) behandelt werden, sondern durch polymorphe Operationen — die Verantwortung wird an die konkreten Typen delegiert.

Anwendung im Projekt

Das System kennt mehrere Turniermodi (Liga, Knockout, Startbildschirm), die sich grundlegend unterscheiden: welche Befehle verfügbar sind, welche Willkommensmeldung erscheint und welche Services benötigt werden.

Ohne Polymorphismus müsste CLI.enterMode() diese Variation durch Fallunterscheidungen selbst auflösen. Stattdessen kennt CLI nur das Interface GameMode und delegiert alle modi-spezifischen Aufrufe polymorph an die konkrete Implementierung.

GameMode — polymorpher Vertrag

```
// GameMode.java – definiert den polymorphen Vertrag
public interface GameMode {
    String name();
    String description();
    void start();
    void onEnter(CommandContext context);
    List<CommandSpec> commands(CommandContext context);
}

// CLI.java – nutzt GameMode rein polymorph, kennt keinen konkreten Typ
public void enterMode(GameMode mode) { // Parameter: Abstraktion, nicht Konkretisierung
    parser.clearRegistry();
    mode.onEnter(parser.context()); // polymorphe Delegation
    parser.registerAll(mode.commands(parser.context())); // polymorphe Delegation
}

// LeagueMode.java – konkrete Ausprägung für Liga-Betrieb
@Override
public void onEnter(CommandContext context) {
    System.out.println("League-Modus aktiv.");
    System.out.println("Start: league create --name ...");
}

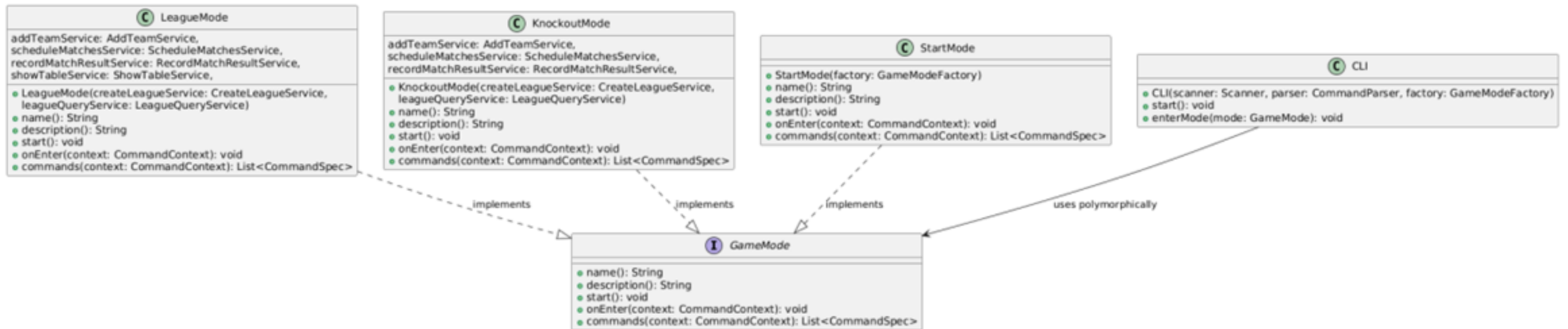
@Override
public List<CommandSpec> commands(CommandContext context) {
    return List.of(
        new CommandSpec("league", new LeagueCommand(createLeagueService), ...),
        new CommandSpec("team", new TeamCommand(addTeamService, ...), ...),
        new CommandSpec("schedule", new ScheduleCommand(...), ...),
        new CommandSpec("match", new MatchCommand(...), ...),
        new CommandSpec("table", new TableCommand(showTableService), ...)
    );
}

// KnockoutMode.java – andere konkrete Ausprägung für KO-Turnier
@Override
public void onEnter(CommandContext context) {
    System.out.println("Knockout-Modus aktiv.");
}

@Override
public List<CommandSpec> commands(CommandContext context) {
    return List.of(
        new CommandSpec("create", new KnockoutLeagueCommand(...), ...),
        new CommandSpec("team", new KnockoutTeamCommand(...), ...),
        new CommandSpec("match", new KnockoutMatchCommand(...), ...)
    );
}
```

CLI nutzt nur das GameMode-Interface; LeagueMode, KnockoutMode und StartMode liefern jeweils ihr eigenes Verhalten.

Polymorphismus — UML



Drei konkrete Implementierungen, eine gemeinsame Abstraktion.

Don't Repeat Yourself — TableRenderer



Ausgangslage (Commit 41a69d7)

Die vollständige Tabellen-Rendering-Logik existierte als private Methode direkt in TableCommand. Gleichzeitig sollte MatchCommand die Tabelle automatisch nach dem letzten Spieltag anzeigen, ohne Refactoring hätte hier die Methode dupliziert werden müssen.

Lösung (Commits e08aaad + 03670b0)

Die Rendering-Logik wurde in zwei Klassen extrahiert: TableRenderer (für die Tabellen-Darstellung) und WinnerRenderer (für die Siegerermittlung).

Ergebnis: Eine einzige Quelle der Wahrheit für das Tabellenformat. Beide Commands zeigen garantiert identische Ausgaben.

Vorher: render()-Logik in TableCommand

```
// TableCommand.java – Zustand vor dem Refactoring (Commit 41a69d7)
public class TableCommand implements Command {

    @Override
    public CommandResult execute(CommandContext ctx, CommandArgs args) {
        // ...
        List<TableRow> rows = showTableService.execute(ctx.getCurrentLeagueId());
        return CommandResult.ok(render(rows)); // ruft private Methode auf
    }

    private String render(List<TableRow> rows) { // Rendering-Logik eingebettet
        if (rows.isEmpty()) {
            return "Keine Daten. Erstelle Teams, Spielplan und Ergebnisse.";
        }
        StringBuilder sb = new StringBuilder();
        sb.append("Tabelle:\n");
        for (TableRow row : rows) {
            sb.append(" - ")
                .append(row.teamName())
                .append(" | ")
                .append("Sp ").append(row.played())
                .append(" W ").append(row.wins())
                .append(" D ").append(row.draws())
                .append(" L ").append(row.losses())
                .append(" | ")
                .append(row.goalsFor()).append(":").append(row.goalsAgainst())
                .append(" | ")
                .append(row.points()).append(" P")
                .append("\n");
        }
        return sb.toString().trim();
    }
}
```

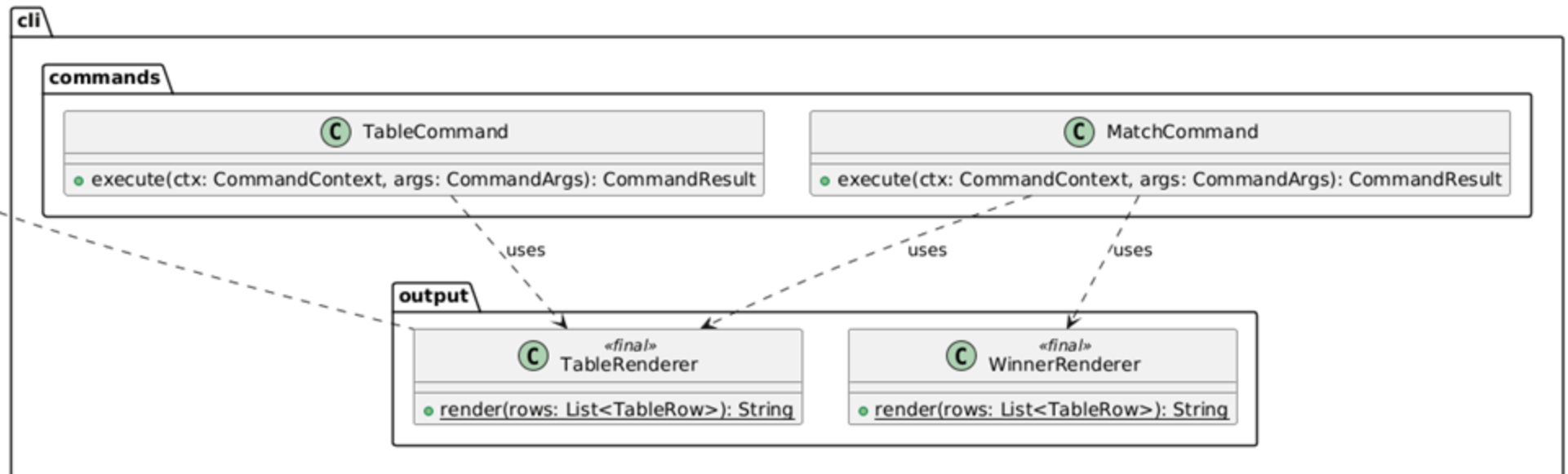
Nachher: extrahierter TableRenderer

```
// TableRenderer.java – neu erstellt in Commit e08aaad
public final class TableRenderer {
    private TableRenderer() {}

    public static String render(List<TableRow> rows) {
        if (rows.isEmpty()) {
            return "Keine Daten. Erstelle Teams, Spielplan und Ergebnisse.";
        }
        StringBuilder sb = new StringBuilder();
        sb.append("Tabelle:\n");
        for (TableRow row : rows) {
            sb.append(" - ").append(row.teamName())
              .append(" | ")
              .append("Sp ").append(row.played())
              .append(" W ").append(row.wins())
              .append(" D ").append(row.draws())
              .append(" L ").append(row.losses())
              .append(" | ")
              .append(row.goalsFor()).append(":").append(row.goalsAgainst())
              .append(" | ")
              .append(row.points()).append(" P\n");
        }
        return sb.toString().trim();
    }
}
```

DRY — UML der Lösung

Vor Refactoring: render()-Logik war private in TableCommand. In MatchCommand wäre sie dupliziert worden.



TableCommand und MatchCommand nutzen denselben TableRenderer.

KAPITEL

05

Unit Tests

10 Tests · ATRIP · Fakes & Mocks

10 Unit Tests — Übersicht



#	Testklasse & Methode	Was wird getestet
1	LeagueWinnerTest.determinesWinnerAfterAllResultsAreRecorded	Nach Eintrag aller Ergebnisse steht das Team mit den meisten Punkten auf Platz 1 der Tabelle. Lions gewinnt alle 3 Spiele → 9 Punkte → Sieger.
2	KnockoutFlowTest.advancesWinnerImmediatelyAndDeterminesChampionForKnockoutBracket	Nach jedem KO-Ergebnis wird der Gewinner sofort in das nächste Match-Slot eingetragen. Nach allen Spielen ist der Turnier-Champion korrekt.
3	KnockoutFlowTest.rejectsDrawInKnockoutMatch	Ein Unentschieden (1:1) im Knockout-Modus wirft eine IllegalArgumentException. Unentschieden sind im KO-System verboten.
4	KnockoutBracketRendererTest.renderBracketAsRoundColumns	Der KnockoutBracketRenderer erzeugt einen ASCII-Bracket mit korrekten Rundenüberschriften ("Halbfinale", "Finale"), Teamnamen, Ergebnissen und Match-IDs.
5	AddTeamServiceTest.rejectsDuplicateTeamName	Das Hinzufügen eines Teams mit einem bereits vorhandenen Namen wirft eine IllegalArgumentException. Die Regel "kein doppelter Teamname" wird erzwungen.
6	AddTeamServiceTest.rejectsAddingTeamAfterMatchesAreScheduled	Nachdem der Spielplan erstellt wurde, wirft das Hinzufügen eines weiteren Teams eine IllegalStateException. Teams können nach Spielplanerstellung nicht mehr geändert werden.
7	ShowTableServiceTest.assignedOnePointEachForDraw	Ein Unentschieden (1:1) gibt beiden beteiligten Teams genau 1 Punkt und 1 Draw. Die Punktberechnung der ShowTableService ist korrekt.
8	RecordMatchResultServiceTest.rejectsScoringAlreadyPlayedMatch	Das nochmalige Eintragen eines Ergebnisses für ein bereits gespieltes Match wirft eine IllegalStateException. Ein Match kann nur einmal bewertet werden.
9	ScheduleMatchesServiceTest.createsCorrectMatchCountForFourTeamRoundRobin	Ein Round-Robin mit 4 Teams erzeugt genau 6 Matches (Formel: $n \cdot (n-1) / 2 = 4 \cdot 3 / 2$). Die Anzahl der erstellten Matches ist korrekt.
10	CreateLeagueServiceTest.persistsLeagueWithCorrectName	CreateLeagueService.execute() ruft repository.save() genau einmal auf. Die gespeicherte Liga trägt den korrekten Namen. Verifiziert via MockLeagueRepository.

Alle 10 implementierten Tests mit Testklasse, Methode und der jeweils geprüften Eigenschaft.

ATRIP — Automatic, Thorough, Professional



Tests laufen ohne manuelles Eingreifen

Edge Cases & Hauptpfade abgedeckt

Reproduzierbar in jeder Umgebung

Tests sind voneinander unabhängig

Sauberer, wartbarer Test-Code

Konkrete Umsetzung:

- Automatic — JUnit 5 + Maven Surefire 3.2.5 in pom.xml; mvn test ohne manuellen Eingriff
- Thorough — Happy Path, Fehlerbehandlung, Business Rules, Berechnungslogik, Persistenz
- Professional — Verb-basierte Methodennamen (rejectsDuplicateTeamName), AAA-Struktur, eigene InMemoryLeagueRepository pro Test, Test-Code nur unter src/test/java

Fake — InMemoryLeagueRepository



Beschreibung

Ein Fake ist ein funktionierendes Ersatzobjekt, das eine vereinfachte Implementierung eines Interfaces bereitstellt, die für die Produktion ungeeignet ist (z. B. kein Datenverlust bei Neustart). InMemoryLeagueRepository implementiert das LeagueRepository-Interface vollständig, speichert jedoch alle Daten ausschließlich im Arbeitsspeicher (LinkedHashMap), statt in einer Datenbank oder Datei.

Einsatzbegründung

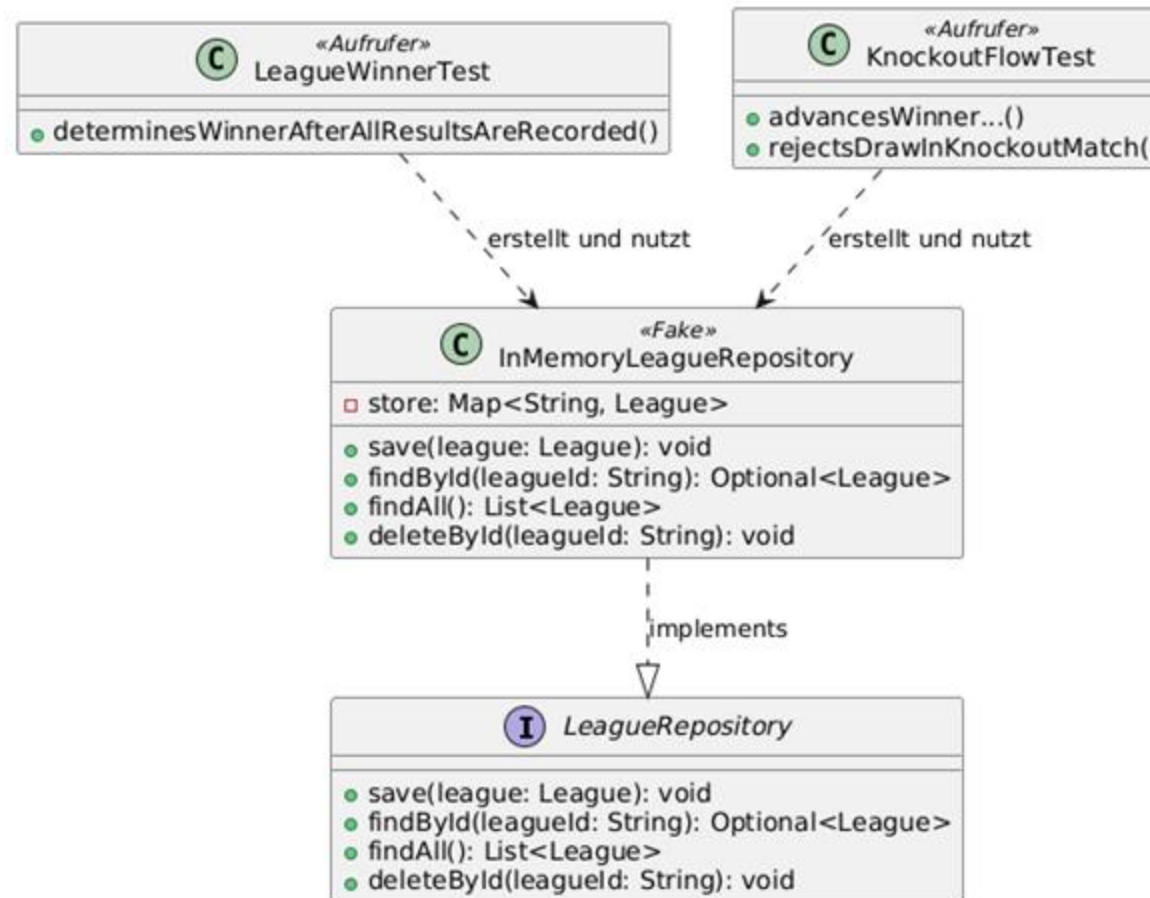
Alle 9 Tests, die Datenbankoperationen benötigen, nutzen diesen Fake. Ohne ihn müsste jeder Test eine echte Persistenzschicht konfigurieren, was Tests langsam, fragil und von externer Infrastruktur abhängig machen würde. Der Fake verhält sich fachlich korrekt (alle Operationen funktionieren), ist aber für Produktion nicht geeignet (Daten gehen beim Neustart verloren).

InMemoryLeagueRepository — Code

```
// InMemoryLeagueRepository.java (Fake – vollständige, aber produktionsuntaugliche Impl.)
public class InMemoryLeagueRepository implements LeagueRepository {
    private final Map<String, League> store = new LinkedHashMap<>();

    @Override public void save(League league)                { store.put(league.id(), league); }
}
    @Override public Optional<League> findById(String leagueId) { return
Optional.ofNullable(store.get(leagueId)); }
    @Override public List<League> findAll()                    { return new ArrayList<>
(store.values()); }
    @Override public void deleteById(String leagueId)          { store.remove(leagueId); }
}
```

InMemoryLeagueRepository — UML



Der Fake wird von zwei Test-Klassen erstellt und genutzt.

Mock — MockLeagueRepository



Beschreibung

Ein Mock zeichnet Interaktionen auf und ermöglicht die Verifikation, ob und wie eine Methode aufgerufen wurde. MockLeagueRepository implementiert LeagueRepository, speichert die übergebene Liga und zählt, wie oft save() aufgerufen wurde. Damit kann in CreateLeagueServiceTest geprüft werden, dass der Service die Liga tatsächlich persistiert — und zwar genau einmal.

Einsatzbegründung

Mit einem Fake kann man nur prüfen, ob Daten vorhanden sind. Ein Mock ermöglicht darüber hinaus die Verifikation des Verhaltens: Hat der Service save() überhaupt aufgerufen? Hat er es nur einmal aufgerufen? Mit welchem Objekt? Diese Fragen sind für einen Unit-Test von CreateLeagueService zentral, da die einzige fachliche Aufgabe des Service genau diese Persistierung ist.

MockLeagueRepository — Code

```
// MockLeagueRepository.java (Mock – aufgezeichnete Interaktionen, keine echte Persistenz)
public class MockLeagueRepository implements LeagueRepository {
    private League savedLeague;
    private int saveCallCount = 0;
    private String lastQueriedId;

    @Override
    public void save(League league) {
        this.savedLeague = league; // Interaktion aufzeichnen
        this.saveCallCount++;      // Aufrufzähler erhöhen
    }

    @Override
    public Optional<League> findById(String leagueId) {
        this.lastQueriedId = leagueId;
        if (savedLeague != null && savedLeague.getId().equals(leagueId)) {
            return Optional.of(savedLeague);
        }
        return Optional.empty();
    }

    @Override public List<League> findAll() { return savedLeague != null ?
List.of(savedLeague) : List.of(); }
    @Override public void deleteById(String leagueId) {}

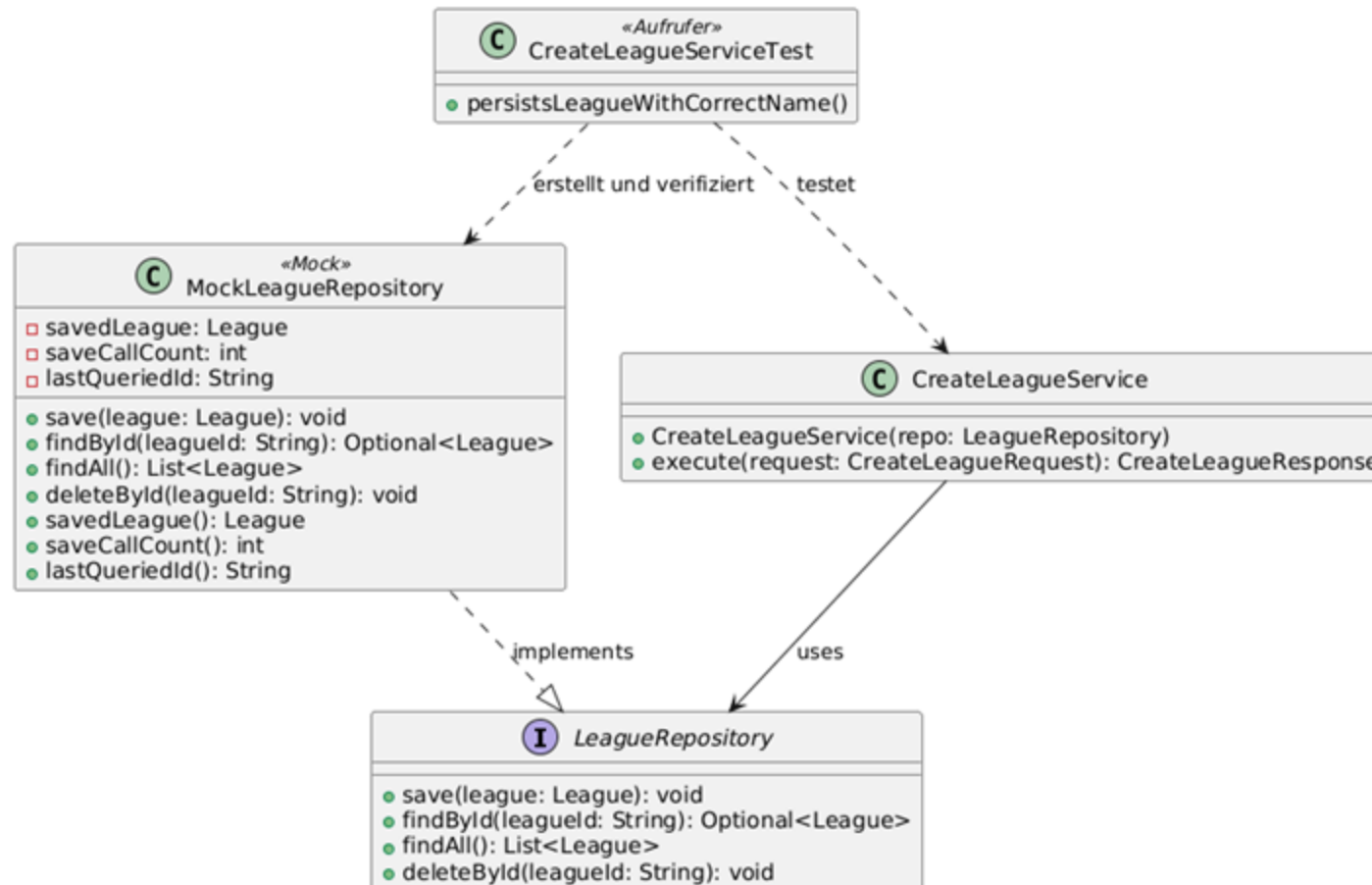
    // Verifikations-Methoden
    public League savedLeague() { return savedLeague; }
    public int saveCallCount() { return saveCallCount; }
    public String lastQueriedId() { return lastQueriedId; }
}

// CreateLeagueServiceTest.java – nutzt den Mock zur Verifikation
@Test
void persistsLeagueWithCorrectName() {
    // Arrange
    MockLeagueRepository mockRepo = new MockLeagueRepository();
    CreateLeagueService service = new CreateLeagueService(mockRepo);

    // Act
    service.execute(new CreateLeagueRequest("Champions Cup"));

    // Assert: save() wurde genau einmal aufgerufen, gespeicherte Liga hat richtigen Namen
    assertEquals(1, mockRepo.saveCallCount());
    assertEquals("Champions Cup", mockRepo.savedLeague().name());
}
```

MockLeagueRepository — UML



Der Mock zeichnet Aufrufe auf und ermöglicht Verifikationen wie `saveCallCount() == 1`.

KAPITEL

06

Domain Driven Design

Ubiquitous Language · Repositories · Aggregates · Entities · Value Objects

Ubiquitous Language



Bezeichnung	Bedeutung	Begründung
League	Ein Turnier-/Wettbewerbsobjekt, das Teams, Spiele und den Spielplan verwaltet. Je nach Format ein Ligabetrieb (Jeder-gegen-Jeden) oder ein KO-Turnier.	Der Begriff „Liga“ bzw. „League“ ist der universelle Fachbegriff, den Sportveranstalter, Trainer und Entwickler gleichermaßen verwenden. Er erscheint direkt als Klassenname <code>League</code> , in CLI-Kommandos (<code>league create</code>) und in Methoden (<code>createLeagueService</code>). Es gibt keine Übersetzung oder Abstraktion zwischen Fachbereich und Code.
Match	Eine einzelne Spielbegegnung zwischen einem Heim- und einem Auswärtsteam, die ein Ergebnis bekommen kann.	„Match“ ist der von allen Beteiligten geteilte Begriff für eine Einzelbegegnung im Sport. Er erscheint als Klassenname <code>Match</code> , in Methoden (<code>recordMatchResult</code> , <code>findMatch</code>), in CLI-Kommandos (<code>match record</code>) und in Tests (<code>KnockoutFlowTest</code>). Niemand würde intern „Game“ oder „Encounter“ sagen.
Score	Das Ergebnis eines Spiels, bestehend aus Heimtor-Anzahl und Auswärtstor-Anzahl.	Im Sportkontext ist „Score“ der etablierte Begriff für das Spielresultat. Er erscheint als Klassenname <code>Score</code> , in CLI-Eingaben (<code>--score 2:1</code>) und in Methoden (<code>recordScore</code>). Die Darstellung als Tupel (<code>home</code> , <code>away</code>) entspricht exakt der fachlichen Vorstellung.
Team	Eine am Turnier teilnehmende Mannschaft, identifiziert durch Name und ID.	„Team“ ist der grundlegende Fachbegriff für eine Teilnehmereinheit in jeden sportlichen Wettbewerb. Er erscheint als Klassenname <code>Team</code> , in Methoden (<code>addTeam</code> , <code>hasTeamName</code>), in CLI-Kommandos (<code>team add</code>) und in Test-Fixtures. Der Begriff wird von Sportveranstaltern, Nutzern und Entwicklern identisch verwendet.

Gemeinsames Vokabular zwischen Fachbereich und Code: *League, Match, Score, Team.*

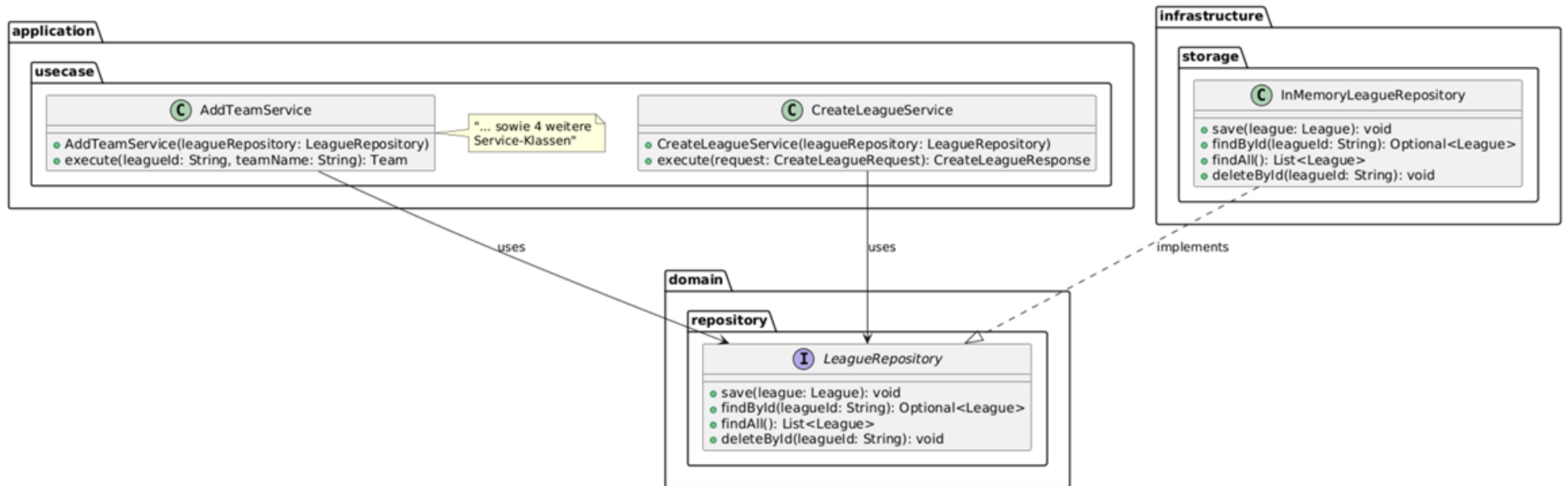
Repositories — LeagueRepository



Ein Repository kommt hier zum Einsatz, weil League das zentrale Aggregat des Systems ist, das über die Laufzeit einer Nutzersitzung hinaus gespeichert und wiederabgerufen werden muss. Alle sechs Use-Case-Services benötigen Zugriff auf gespeicherte Ligen (laden per `findById`, speichern per `save`).

Das Interface liegt bewusst im Domain-Layer, sodass der Domänekern keinerlei Kenntnis über die konkrete Speichertechnik hat. Dadurch lässt sich `InMemoryLeagueRepository` jederzeit durch eine Datei- oder Datenbank-Implementierung ersetzen, ohne eine einzige Zeile in Domain oder Application zu ändern.

LeagueRepository — UML



Interface im Domain-Layer; konkrete Implementierung in Infrastructure; Nutzung aus Application.

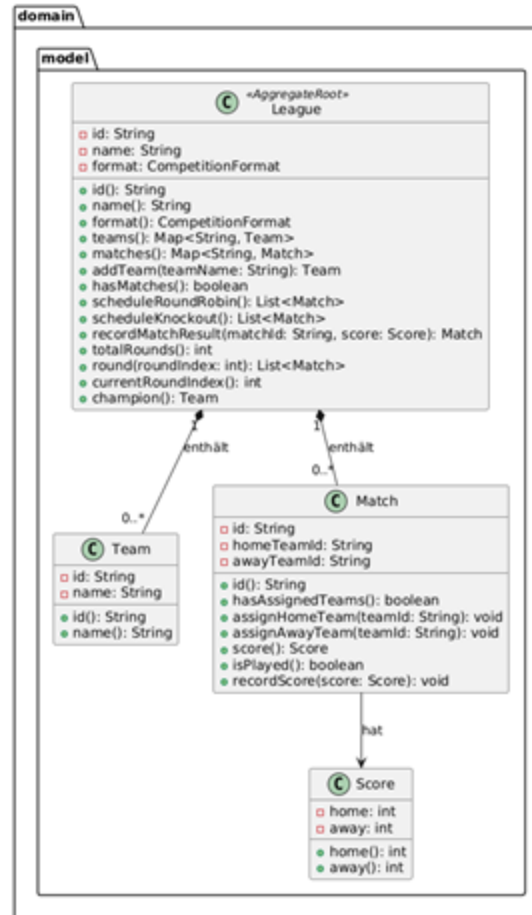
Aggregates — League als Aggregate Root



League ist der Aggregate Root. Es bildet das zentrale Aggregat, das alle zusammengehörenden Objekte (Team, Match, Score) kapselt und deren Konsistenz erzwingt.

Kein externer Code modifiziert Team- oder Match-Objekte direkt — alle Operationen laufen über League-Methoden.

Aggregate League — UML



League umschließt Team, Match und Score; Zugriff erfolgt nur über League.

Entities — Match



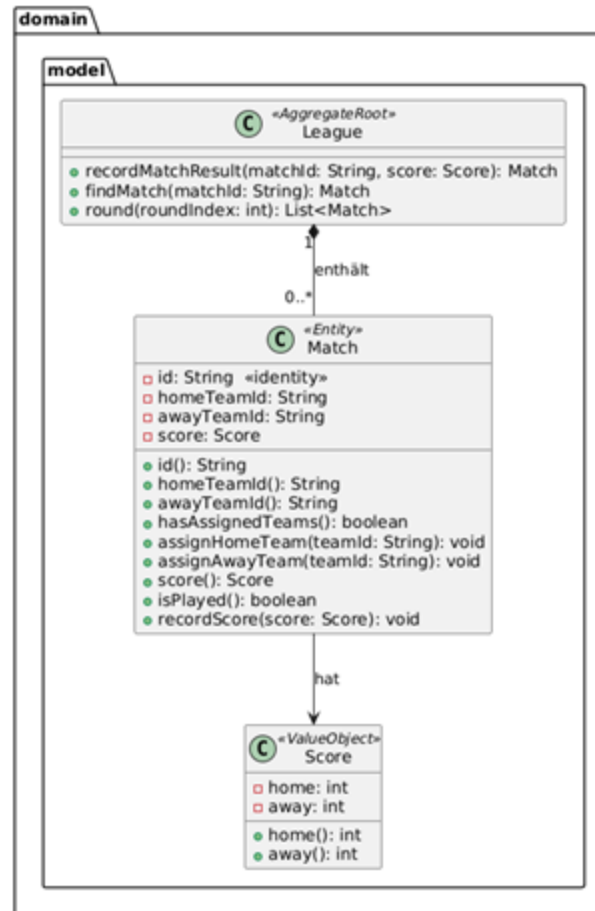
Match ist eine Entity. Jedes Spiel besitzt eine stabile, eindeutige Identität (id, z. B. „M1“, „M3“), anhand derer es über seinen gesamten Lebenszyklus hinweg identifiziert wird — unabhängig davon, welche Teams zugewiesen werden oder ob bereits ein Ergebnis eingetragen wurde.

Begründung

Match ist eine Entity, weil seine Identität und nicht seine Attributwerte entscheidend sind. Zwei Spiele mit demselben Heim- und Auswärtsteam sind dennoch verschiedene Matches, wenn ihre IDs unterschiedlich sind. In einer Liga spielen FC Bayern und Dortmund mehrfach gegeneinander — jedes dieser Spiele muss separat identifizierbar bleiben.

Außerdem durchläuft Match einen definierten Lebenszyklus: Es wird zunächst ohne Teams erstellt (Knockout-Platzhalter), erhält dann via `assignHomeTeam/assignAwayTeam` seine Teilnehmer und schließlich via `recordScore` sein Ergebnis.

Entity Match — UML



Match besitzt eine «identity»-id und durchläuft einen definierten Lebenszyklus.

Value Objects — Score

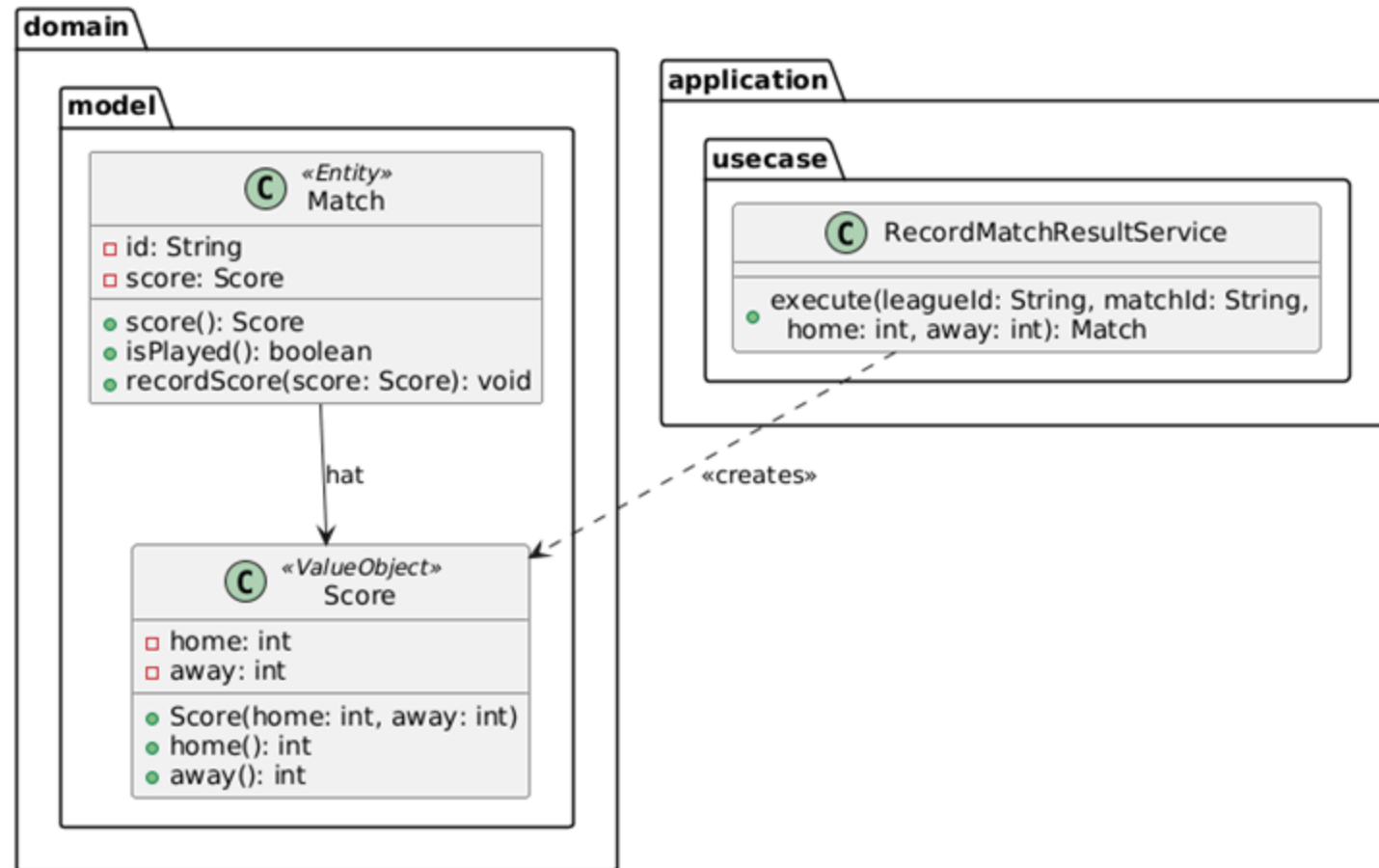


Score ist ein Value Object. Es repräsentiert das Ergebnis eines Spiels als Paar aus Heim- und Auswärtstor-Anzahl. Es besitzt keine eigene Identität — zwei Score(2, 1)-Objekte sind fachlich absolut gleichwertig.

Score erfüllt alle Kriterien eines Value Objects:

- Keine Identität — zwei Scores mit gleichen Werten sind austauschbar
- Unveränderlich — die Werte werden im Konstruktor gesetzt und nie geändert
- Wertbasierte Gleichheit — definiert über home und away, nicht über Identität
- Selbstvalidierend — ein Score kapselt sein eigenes Format und seine Regeln

Value Object Score — UML



Match (Entity) hat einen Score (Value Object); Score wird vom Application-Service erzeugt.

KAPITEL

07

Refactoring

Code Smells · Refactorings

Code Smell 1: Long Parameter List — LeagueMode



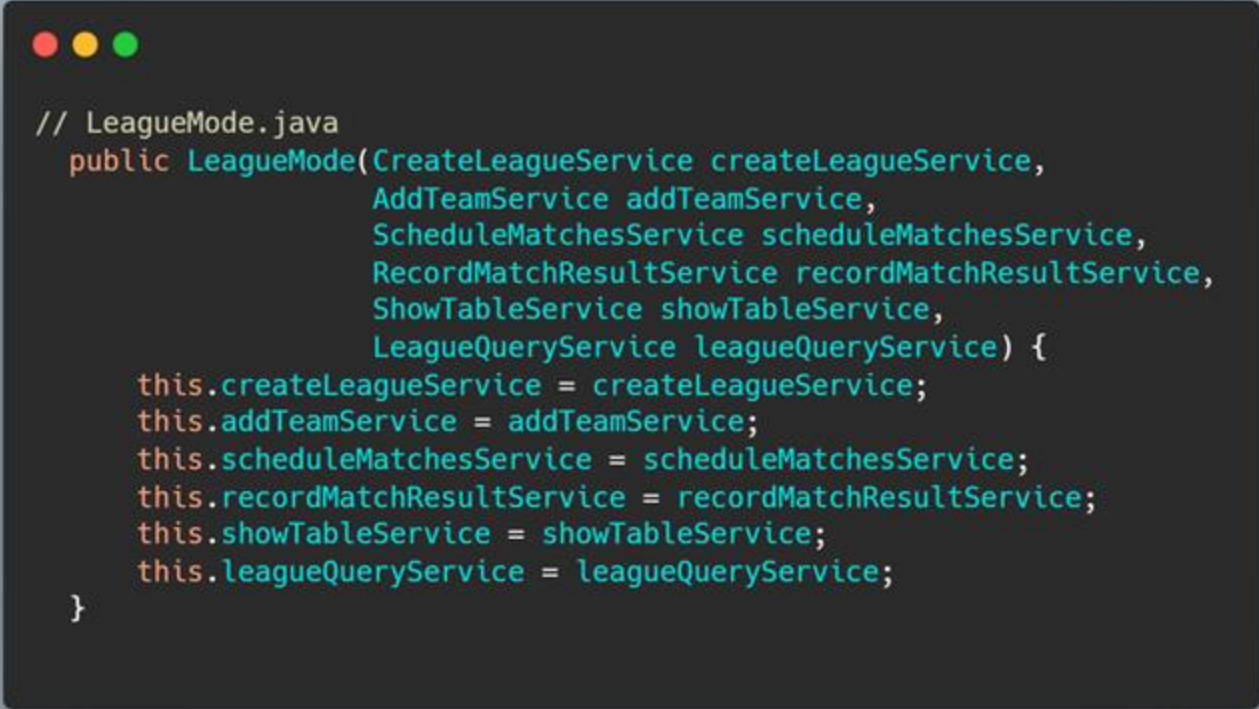
Problem

Der Konstruktor von LeagueMode nimmt 6 Parameter entgegen — alle Use-Case-Services der Anwendungsschicht. Diese „Long Parameter List“ macht den Konstruktor schwer lesbar und Aufrufstellen unübersichtlich.

Lösungsweg — Introduce Parameter Object

Alle zusammengehörenden Services werden in einem Parameter-Objekt LeagueServices gebündelt. Der Konstruktor nimmt dann nur noch ein einziges Objekt entgegen und entpackt sich seine Abhängigkeiten daraus.

Long Parameter List — Vorher



```
// LeagueMode.java
public LeagueMode(CreateLeagueService createLeagueService,
                  AddTeamService addTeamService,
                  ScheduleMatchesService scheduleMatchesService,
                  RecordMatchResultService recordMatchResultService,
                  ShowTableService showTableService,
                  LeagueQueryService leagueQueryService) {
    this.createLeagueService = createLeagueService;
    this.addTeamService = addTeamService;
    this.scheduleMatchesService = scheduleMatchesService;
    this.recordMatchResultService = recordMatchResultService;
    this.showTableService = showTableService;
    this.leagueQueryService = leagueQueryService;
}
```

6 Parameter im Konstruktor — schwer lesbar, schwer zu erweitern.

Long Parameter List — Nachher

```
// Pseudo-Code: neues Parameter-Objekt
public record LeagueServices(
    CreateLeagueService createLeague,
    AddTeamService addTeam,
    ScheduleMatchesService scheduleMatches,
    RecordMatchResultService recordMatch,
    ShowTableService showTable,
    LeagueQueryService leagueQuery
) {}

// Vereinfachter Konstruktor
public LeagueMode(LeagueServices services) {
    this.createLeagueService = services.createLeague();
    this.addTeamService      = services.addTeam();
    // ...
}
```

Ein Parameter-Objekt `LeagueServices` bündelt alle zusammengehörenden Abhängigkeiten.

Code Smell 2: Long Method — MatchCommand.execute()

Problem

Die Methode MatchCommand.execute() umfasst 65 Zeilen und behandelt inline mehrere Verantwortlichkeiten: Validierung, Aktionsunterscheidung, Score-Parsing, Rundenübergangserkennung und Tabellenrendering. Die record-Verzweigung allein ist ~40 Zeilen lang.

Lösungsweg — Extract Method

Der gesamte record-Block wird in eine eigene private Methode handleRecord() extrahiert. execute() bleibt als kurzer Dispatcher bestehen, jede Aktion bekommt ihre eigene klar benannte Methode.

Long Method — Vorher

```
// MatchCommand.java - execute() (gekürzt, zeigt die Länge)
@Override
public CommandResult execute(CommandContext ctx, CommandArgs args) {
    String action = args.pos(0);
    if (action == null || action.equalsIgnoreCase("help")) { ... } // Zeile 34-36
    if (ctx.getCurrentLeagueId() == null) { ... } // Zeile 38-40
    if (action.equalsIgnoreCase("list")) { ... } // Zeile 42-44

    if (action.equalsIgnoreCase("record")) { // Zeile 46
        String id = args.getOption("id");
        String score = args.getOption("score");
        if (id == null || id.isBlank()) { ... } // Validierung
        if (score == null || score.isBlank()) { ... } // Validierung
        int[] parsed = parseScore(score);
        if (parsed == null) { ... }
        try {
            League before = leagueQueryService.byId(...);
            int beforeRound = before.currentRoundIndex();
            recordMatchResultService.execute(...);
            League after = leagueQueryService.byId(...);
            int afterRound = after.currentRoundIndex();

            StringBuilder sb = new StringBuilder();
            sb.append("Ergebnis gespeichert.");
            if (afterRound == -1) { // alle Runden fertig
                List<TableRow> rows = showTableService.execute(...);
                sb.append(TableRenderer.render(rows));
                sb.append(WinnerRenderer.render(rows));
                return CommandResult.ok(sb.toString().trim());
            }
            if (beforeRound != -1 && afterRound != beforeRound) { // Rundenübergang
                sb.append("Runde abgeschlossen...");
                appendRoundMatches(sb, after, afterRound);
                ...
                return CommandResult.ok(sb.toString());
            }
            ...
            return CommandResult.ok(sb.toString());
        } catch (IllegalArgumentException e) { ... }
    }
    return CommandResult.invalid("Unbekannte Aktion. " + helpText());
}
```

65 Zeilen, 5 Verantwortlichkeiten in einer Methode.

Long Method — Nachher

```
// Pseudo-Code nach Extract Method
@Override
public CommandResult execute(CommandContext ctx, CommandArgs args) {
    String action = args.pos(0);
    if (action == null || action.equalsIgnoreCase("help")) {
        return CommandResult.ok(helpText());
    }
    if (ctx.getCurrentLeagueId() == null) {
        return CommandResult.invalid("Keine aktive Liga.");
    }
    if (action.equalsIgnoreCase("list")) {
        return listMatches(ctx.getCurrentLeagueId(), args.getOption("round"));
    }
    if (action.equalsIgnoreCase("record")) {
        return handleRecord(ctx, args.getOption("id"), args.getOption("score"));
    }
    return CommandResult.invalid("Unbekannte Aktion. " + helpText());
}

// Extrahierte Methode – klar benannt, eine Aufgabe
private CommandResult handleRecord(CommandContext ctx, String id, String scoreStr) {
    if (id == null || id.isBlank()) { ... }
    if (scoreStr == null || scoreStr.isBlank()) { ... }
    int[] parsed = parseScore(scoreStr);
    if (parsed == null) { ... }
    try {
        // ... Logik nur noch hier ...
    } catch (IllegalArgumentException e) { ... }
}
```

execute() ist ein schlanker Dispatcher; die Detail-Logik liegt in handleRecord().

Refactoring 1: Extract Method — GameModeSelectionService



Commit 3340295

Vorher

GameModeSelectionService hatte eine einzige öffentliche Methode `execute()`, die zwei Aufgaben in einem Durchlauf erledigte: die Modusauswahl auf der Konsole ausgeben UND die zugehörigen `CommandSpec`-Objekte zurückgeben. Außerdem rief sie `cli.readInput()` auf und besaß damit eine direkte Abhängigkeit auf CLI.

Nachher

Die Methode `execute()` wurde in zwei klar benannte Methoden `printSelection()` und `selectionCommands()` aufgeteilt. Die Abhängigkeit auf CLI entfiel komplett.

Begründung

`execute()` verletzte SRP: Ausgabe und Command-Erzeugung sind orthogonale Aufgaben. StartMode muss `onEnter()` (Ausgabe) und `commands()` (Befehlsliste) getrennt aufrufen — das war mit einer einzigen Methode nicht abbildbar.

Refactoring 1 — Code vorher

```
// GameModeSelectionService.java – VOR dem Refactoring (Commit 41a69d7)
public class GameModeSelectionService {
    /**
     * Verfügbare GameModes abfragen
     * User-Auswahl entgegennehmen
     * Passenden GameMode erzeugen
     * Kontrolle an den GameMode übergeben
     */
    private final CLI cli;           // Abhängigkeit auf CLI
    private final GameModeFactory factory;

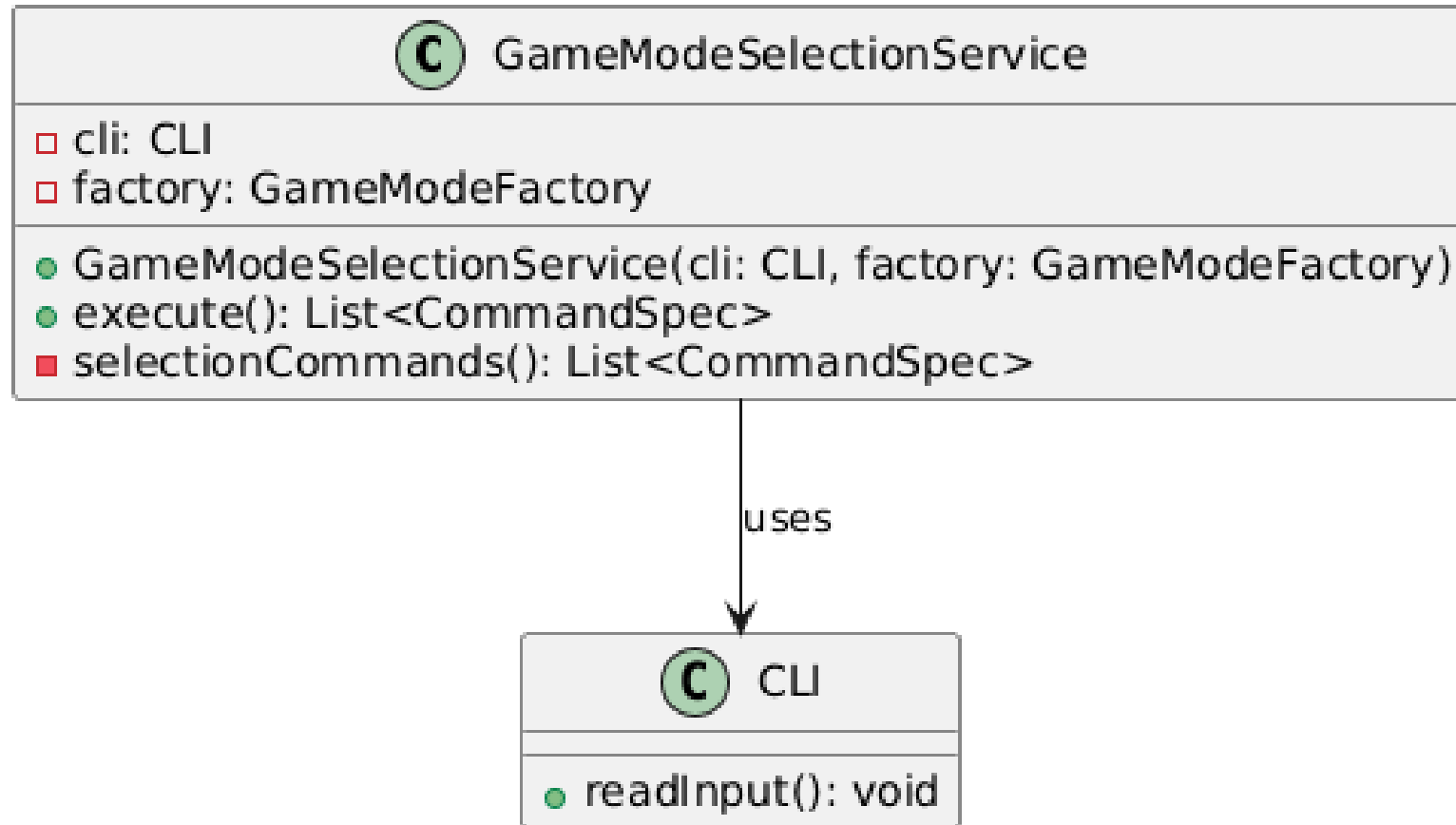
    public GameModeSelectionService(CLI cli, GameModeFactory factory) {
        this.cli = cli;
        this.factory = factory;
    }

    public List<CommandSpec> execute() {
        List<CommandSpec> commands = selectionCommands(); // Befehle laden
        System.out.println("Bitte wähle einen Spielmodus:\n");
        int index = 1;
        for (GameModeType type : GameModeType.values()) {
            String availability = factory.isSupported(type) ? "" : " (noch nicht verfügbar)";
            System.out.println(index++ + " " + type.displayName() + " – " + type.description() +
                                availability);
        }
        System.out.println("\nEingabe: Nummer oder Name des Spielmodus");
        cli.readInput();           // Liest direkt Input – Kopplung an CLI
        return commands;           // Gibt Commands zurück
    }

    private List<CommandSpec> selectionCommands() { ... }
}
```

execute() vermischt Ausgabe, CLI-Eingabe-Lesen und Command-Rückgabe.

Refactoring 1 — UML vorher



Direkte Abhängigkeit auf CLI durch `readInput()`.

Refactoring 1 — Code nachher

```
// GameModeSelectionService.java – NACH dem Refactoring (Commit 3340295)
public class GameModeSelectionService {
    private final GameModeFactory factory; // CLI-Abhängigkeit entfernt

    public GameModeSelectionService(GameModeFactory factory) {
        this.factory = factory;
    }

    public void printSelection() { // Extrahiert: nur noch Ausgabe
        System.out.println("Bitte waehle einen Spielmodus:\n");
        int index = 1;
        for (GameModeType type : GameModeType.values()) {
            String availability = factory.isSupported(type) ? "" : " (noch nicht verfuegbar)";
            System.out.println(index++ + " " + type.displayName() + " - " + type.description() +
availability);
        }
        System.out.println("\nEingabe: Nummer oder Name des Spielmodus");
    }

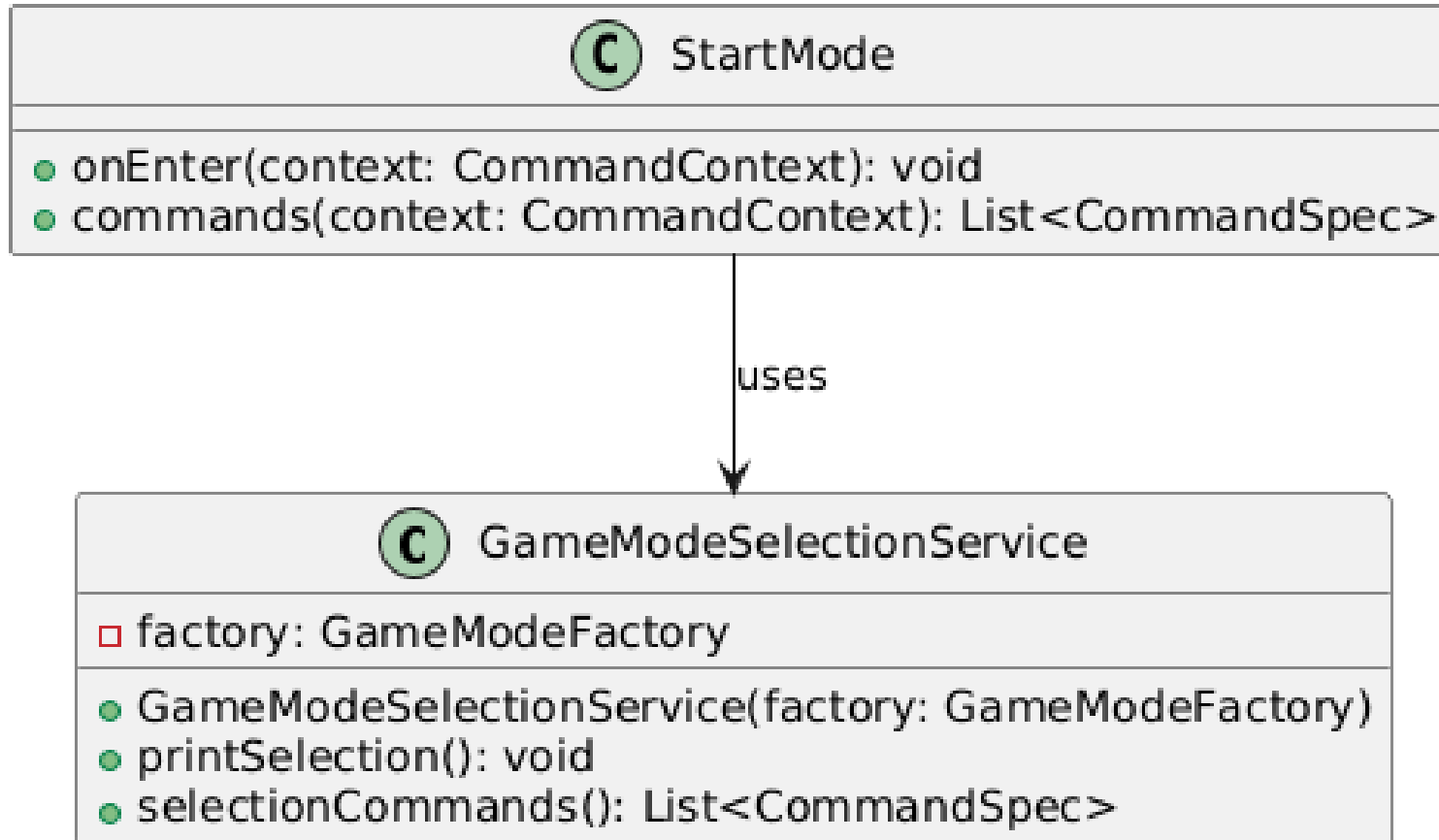
    public List<CommandSpec> selectionCommands() { // Extrahiert: nur noch Commands
        return List.of(...);
    }
}

Der Aufrufer StartMode ruft beide Methoden getrennt auf:
// StartMode.java – nach Refactoring
@Override
public void onEnter(CommandContext context) {
    selectionService.printSelection(); // nur Ausgabe
}

@Override
public List<CommandSpec> commands(CommandContext context) {
    return selectionService.selectionCommands(); // nur Commands
}
```

Zwei Methoden mit jeweils einer Aufgabe; CLI-Abhängigkeit eliminiert.

Refactoring 1 — UML nachher



StartMode nutzt printSelection() und selectionCommands() getrennt.

Refactoring 2: Extract Class — TableRenderer



Commits 03670b0 + e08aaad

Vorher

Die Rendering-Logik für die Ligatabelle befand sich als private Methode `render()` direkt in `TableCommand`. Als `MatchCommand` die Tabelle ebenfalls automatisch nach dem letzten Spieltag anzeigen sollte, wäre eine Duplikation dieser 17-zeiligen Methode unvermeidlich gewesen.

Durchgeführtes Refactoring

Die `render()`-Methode und die gesamte Rendering-Verantwortung wurden in eine neue Klasse `TableRenderer` extrahiert. Parallel entstand `WinnerRenderer` für die Siegerermittlung.

Begründung

`TableCommand` trug eine Verantwortung, die nicht zu ihm gehörte: das Formatieren von Tabellendaten ist keine CLI-Kommandologik, sondern eine eigenständige Darstellungsaufgabe. Ergebnis: keine Duplikation, eine einzige Quelle der Wahrheit für das Tabellenformat, beide Commands zeigen garantiert identische Ausgaben.

Refactoring 2 — Code vorher

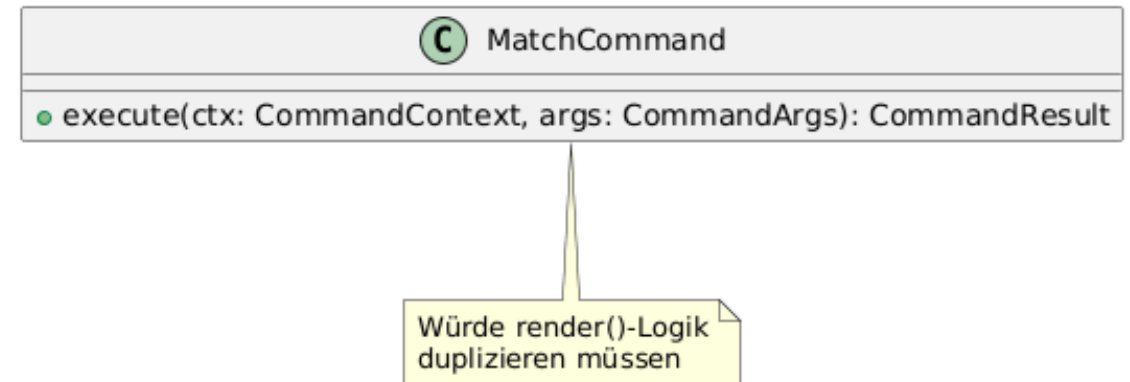
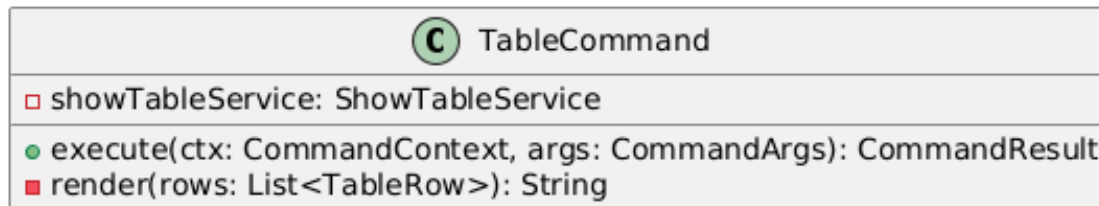
```
// TableCommand.java – VOR dem Refactoring (Commit 41a69d7)
public class TableCommand implements Command {
    private final ShowTableService showTableService;

    @Override
    public CommandResult execute(CommandContext ctx, CommandArgs args) {
        List<TableRow> rows = showTableService.execute(ctx.getCurrentLeagueId());
        return CommandResult.ok(render(rows)); // ruft lokale Methode
    }

    private String render(List<TableRow> rows) { // Rendering-Logik in TableCommand
        if (rows.isEmpty()) { return "Keine Daten..."; }
        StringBuilder sb = new StringBuilder();
        sb.append("Tabelle:\n");
        for (TableRow row : rows) {
            sb.append(" - ").append(row.teamName())
              .append(" | Sp ").append(row.played())
              .append(" W ").append(row.wins())
              // ...
              .append(row.points()).append(" P\n");
        }
        return sb.toString().trim();
    }
}
```

render()-Logik liegt als private Methode in TableCommand.

Refactoring 2 — UML vorher



MatchCommand würde render() duplizieren müssen.

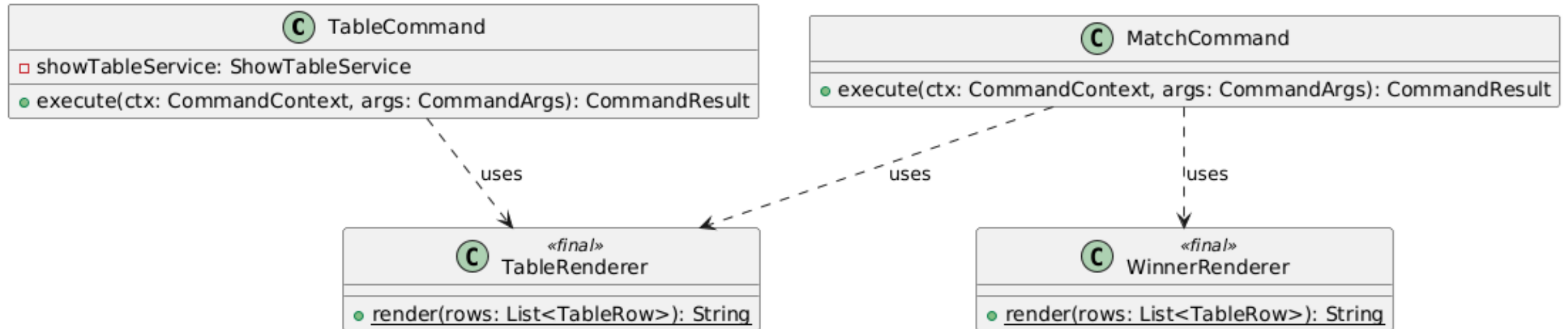
Refactoring 2 — Code nachher

```
// TableRenderer.java – NACH dem Refactoring (Commit e08aaad) – neue Klasse
public final class TableRenderer {
    private TableRenderer() {}

    public static String render(List<TableRow> rows) {
        if (rows.isEmpty()) { return "Keine Daten..."; }
        StringBuilder sb = new StringBuilder();
        sb.append("Tabelle:\n");
        for (TableRow row : rows) {
            sb.append(" - ").append(row.teamName())
              .append(" | Sp ").append(row.played())
              // ...
              .append(row.points()).append(" P\n");
        }
        return sb.toString().trim();
    }
}
```

Eigene Klasse TableRenderer mit static render(); TableCommand wird drastisch kürzer.

Refactoring 2 — UML nachher



TableCommand und MatchCommand nutzen denselben TableRenderer und WinnerRenderer.

KAPITEL

08

Entwurfsmuster

Adapter · MVC

Muster 1: Adapter (Strukturmuster)



Beschreibung

Der Adapter konvertiert die Schnittstelle einer Klasse in eine andere Schnittstelle, die der Client erwartet. Er ermöglicht die Zusammenarbeit von Klassen, deren Interfaces sonst nicht kompatibel wären.

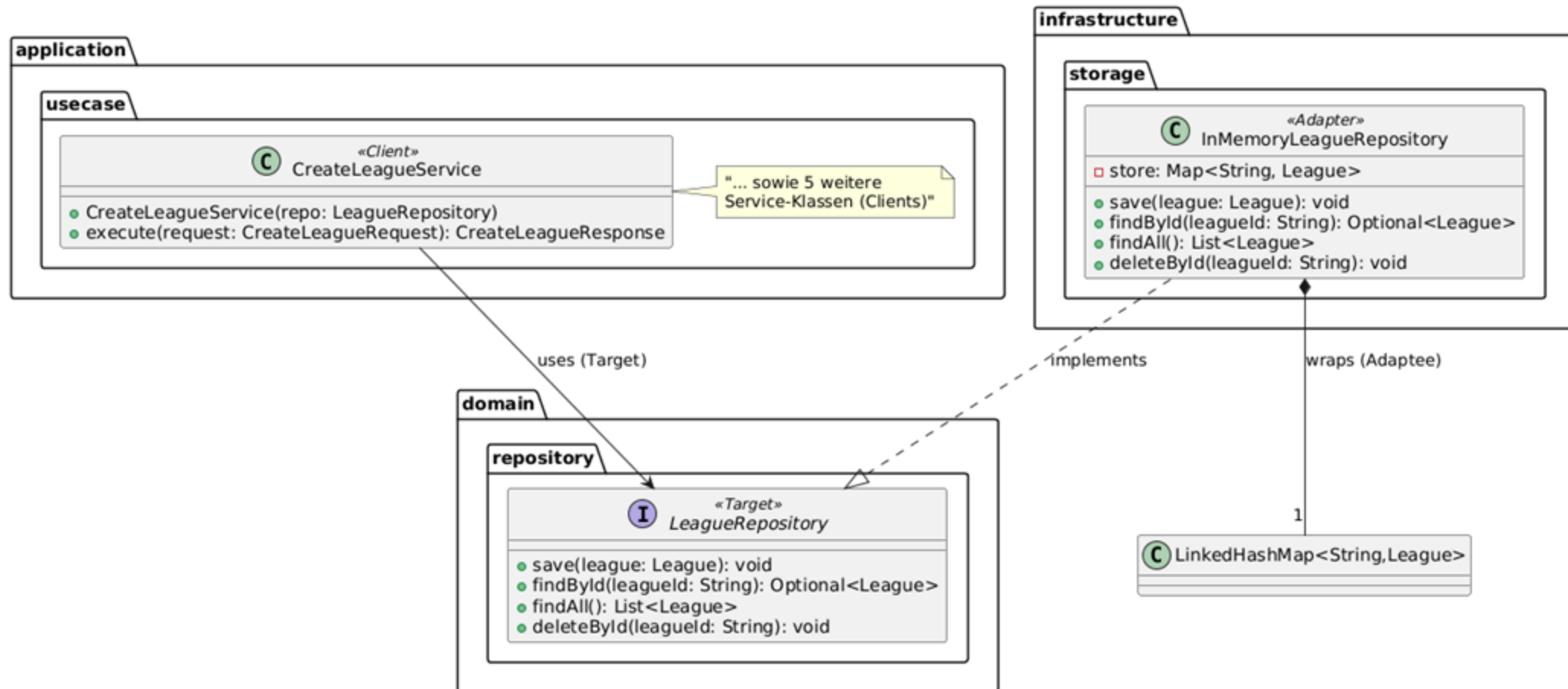
Anwendung im Projekt

Im Projekt adaptiert `InMemoryLeagueRepository` die Java-Standarddatenstruktur `LinkedHashMap<String, League>` (Adaptee) auf das Domain-Interface `LeagueRepository` (Target), das die Anwendungsschicht erwartet.

Begründung

Die Domain kennt keine `HashMap`, `ArrayList` oder sonstige Java-Infrastruktur sie definiert nur, was ein `Repository` leisten muss. `LinkedHashMap` erfüllt diese Schnittstelle nicht direkt: ihre Methoden heißen `put`, `get`, `values`, `remove`. Der Adapter `InMemoryLeagueRepository` schließt genau diese Lücke und übersetzt jede Domain-Operation in die entsprechende Map-Operation. Soll die Persistenz auf eine Datei oder Datenbank umgestellt werden, genügt eine neue Adapter-Implementierung kein Client muss angepasst werden.

Adapter — UML



InMemoryLeagueRepository (Adapter) wickelt LinkedListHashMap (Adaptee) und implementiert LeagueRepository (Target).

Adapter — Code

```
// InMemoryLeagueRepository.java – Adapter übersetzt alle Methodenaufrufe
public class InMemoryLeagueRepository implements LeagueRepository {
    private final Map<String, League> store = new LinkedHashMap<>(); // Adaptee

    @Override
    public void save(League league) {
        store.put(league.id(), league); // target.save() → adaptee.put()
    }

    @Override
    public Optional<League> findById(String leagueId) {
        return Optional.ofNullable(store.get(leagueId)); // target.findById() → adaptee.get()
    }

    @Override
    public List<League> findAll() {
        return new ArrayList<>(store.values()); // target.findAll() → adaptee.values()
    }

    @Override
    public void deleteById(String leagueId) {
        store.remove(leagueId); // target.deleteById() → adaptee.remove()
    }
}
```

Jede Domain-Operation wird auf eine Map-Operation übersetzt.

Muster 2: MVC (Architekturmuster)



Beschreibung

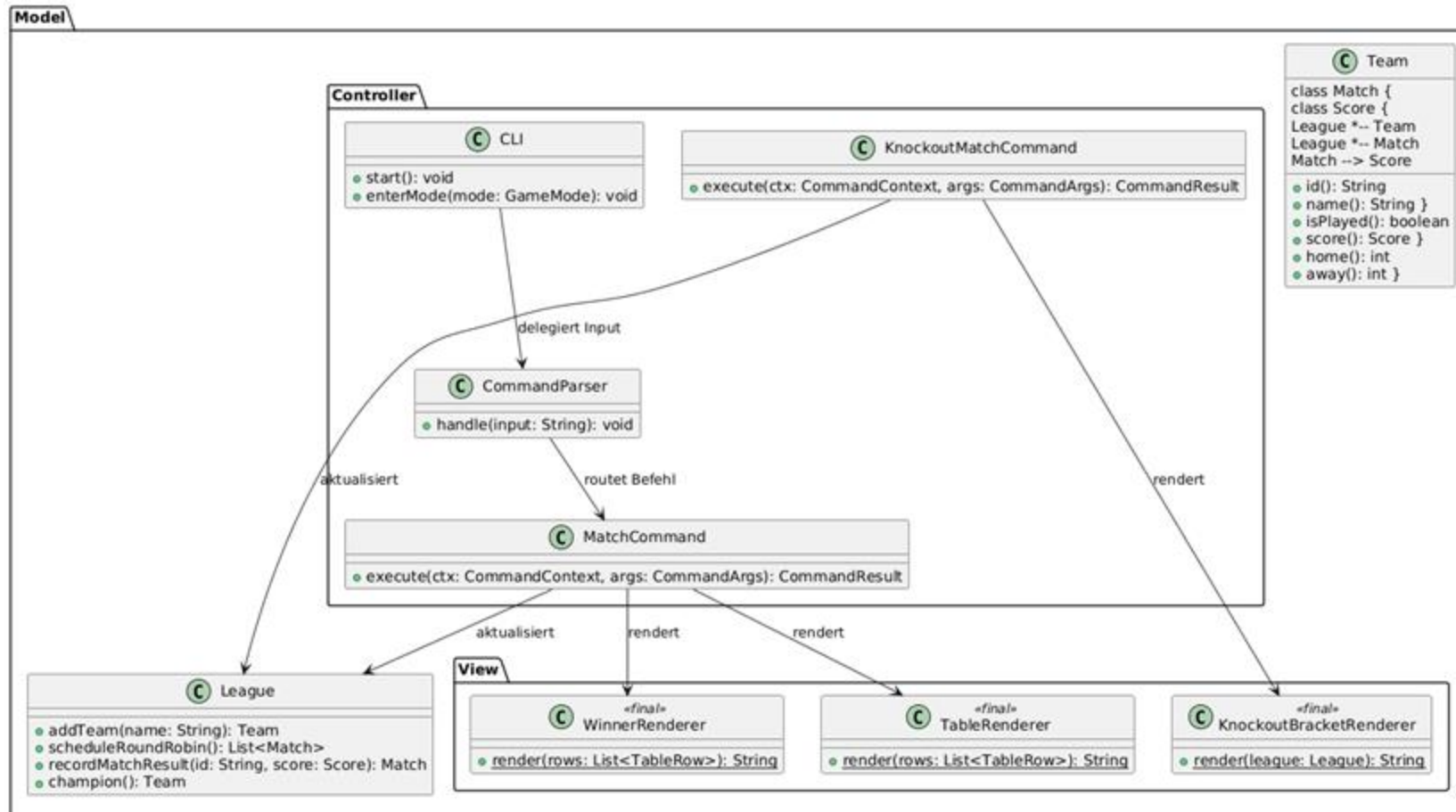
MVC trennt eine Anwendung in drei klar voneinander abgegrenzte Schichten: Model (Daten und Domänenlogik), View (Darstellung) und Controller (Eingabeverarbeitung und Koordination). Änderungen in einer Schicht haben minimale Auswirkungen auf die anderen.

Anwendung im Projekt

- Model — League, Team, Match: enthält ausschließlich Domänenlogik und Zustand. League weiß nicht, ob seine Daten als ASCII-Tabelle oder JSON ausgegeben werden.
- View — Renderer-Klassen kennen nur Daten-DTOs oder Domain-Objekte, niemals Commands oder CLI-Strukturen. Sie sind rein darstellend.
- Controller — CLI + Commands verarbeiten Nutzereingaben, koordinieren Model-Updates über Services und geben aufbereitete Daten an die View weiter.

Die Trennung ermöglicht es, die CLI-Schicht vollständig auszutauschen (z. B. durch eine REST-API), ohne Model oder View zu berühren.

MVC — UML



Drei klar voneinander getrennte Schichten: Model (League, Team), Controller (CLI, Commands), View (Renderer).

MVC — Code

```
// Controller: MatchCommand.execute() – empfängt Input, koordiniert Model und View
if (action.equalsIgnoreCase("record")) {
    // 1. Model aktualisieren (via Service)
    recordMatchResultService.execute(ctx.getCurrentLeagueId(), id, parsed[0], parsed[1]);

    // 2. Model lesen (für Anzeige)
    List<TableRow> rows = showTableService.execute(ctx.getCurrentLeagueId());

    // 3. View aufrufen – Controller übergibt Daten, View rendert
    sb.append(TableRenderer.render(rows)); // View kennt Controller nicht
    sb.append(WinnerRenderer.render(rows)); // View kennt Model nur als Daten-DTO
}

// Controller: KnockoutMatchCommand.execute() – analoges Muster
recordMatchResultService.execute(ctx.getCurrentLeagueId(), id, ...);
League league = leagueQueryService.byId(ctx.getCurrentLeagueId());
return CommandResult.ok(KnockoutBracketRenderer.render(league)); // View erhält Model-Objekt

// View: TableRenderer – reine Darstellungslogik, kennt weder CLI noch Commands
public final class TableRenderer {
    public static String render(List<TableRow> rows) { // empfängt nur Daten
        // ... formatiert und gibt String zurück
        // kein Import von CLI, Command, League, Service
    }
}

// View: KnockoutBracketRenderer – liest League-Daten, kennt keine Controller
public final class KnockoutBracketRenderer {
    public static String render(League league) { // liest Domain-Objekt
        // ... rendert ASCII-Bracket
        // kein Import von CLI, Command, Service
    }
}

// Model: League – reine Domänenlogik, kennt weder View noch Controller
public class League {
    public Match recordMatchResult(String matchId, Score score) { ... }
    public Team champion() { ... }
    // kein Import von Renderer, CLI oder Command
}
```

MatchCommand (Controller) aktualisiert Model über Services und delegiert Darstellung an die View.

VIELEN DANK

Fragen & Diskussion

LeagueMaster · Programmentwurf-Protokoll

Leon Rother · Moritz Mürle · 03.05.2026